



POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

Praca dyplomowa inżynierska

na kierunku Informatyka
na specjalności Inżynieria oprogramowania

Integracja komponentu renderującego obiekty 3D z internetowym serwisem aukcyjnym

Integration of a component rendering 3D objects with an online auction service

Konrad Tomasz Nowak

99640

Maciej Ołdakowski

99648

Patryk Kamil Nowacki

99638

Promotor Dr Marcin Barszcz

Lublin 2025

Streszczenie

Jednym z kluczowych wyzwań w handlu internetowym, w szczególności w sektorze aukcji online, jest zapewnienie kupującym możliwości dokładnego zapoznania się ze stanem oferowanego przedmiotu. Tradycyjne platformy aukcyjne opierają się głównie na zdjęciach i opisach tekstowych, które nie zawsze w pełni oddają rzeczywisty stan produktu. Celem niniejszej pracy było zaprojektowanie i implementacja systemu informatycznego integrującego komponent renderujący modele trójwymiarowe z internetowym serwisem aukcyjnym.

Komponent renderujący został zaimplementowany w języku C++ z wykorzystaniem API WebGPU, który umożliwia niskopoziomą komunikację z procesorem graficznym. Kod skompilowano do formatu WebAssembly przy użyciu narzędzia Emscripten. Część kliencka serwisu aukcyjnego została zrealizowana z wykorzystaniem biblioteki React, natomiast część serwerowa implementuje logikę biznesową typową dla platform aukcyjnych. Integracja komponentu renderującego z serwisem została zrealizowana przy użyciu wzorca Provider oraz mechanizmu React Portals.

W wyniku pracy powstał w pełni funkcjonalny system umożliwiający interaktywną wizualizację trójwymiarowych modeli przedmiotów aukcyjnych bezpośrednio w przeglądarce internetowej. Osiągnięto wydajność renderowania porównywalną z aplikacjami natywnymi przy zachowaniu kompatybilności z wszystkimi współczesnymi przeglądarkami wspierającymi WebGPU. Cel pracy został w pełni zrealizowany.

Abstract

One of the key challenges in online commerce, particularly in the online auction sector, is to provide buyers with the opportunity to thoroughly familiarise themselves with the condition of the item being offered. Traditional auction platforms rely mainly on photographs and text descriptions, which do not always fully reflect the actual condition of the product. The aim of this thesis was to design and implement an IT system integrating a component that renders three-dimensional models with an online auction service.

The rendering component was implemented in C++ using the WebGPU API, which enables low-level communication with the graphics processor. The code was compiled to WebAssembly format using the Emscripten toolchain. The client side of the auction service was implemented using the React library, while the server side implements business logic typical for auction platforms. The integration of the rendering component with the service was achieved using the Provider pattern and the React Portals mechanism.

The result of this thesis is a fully functional system that enables interactive visualisation of three-dimensional models of auction items directly in a web browser. Rendering performance comparable to native applications has been achieved while maintaining compatibility with all modern browsers that support WebGPU. The aim of the thesis has been fully achieved.

Spis treści

1	Wstęp	4
2	Cel i zakres pracy	5
2.1	Cel pracy	5
2.2	Zakres pracy	5
3	Analiza rynku	7
3.1	allegro.pl	7
3.2	amazon.pl	7
3.3	erli.pl	8
3.4	Wnioski	8
4	Użyte technologie i narzędzia	9
4.1	Komponent renderujący	9
4.2	System aukcyjny - Frontend	10
4.3	System aukcyjny - Backend	11
5	Projekt aplikacji	12
5.1	Udokumentowanie analizy i udokumentowanie rozpoznanych procesów biznesowych	12
5.1.1	Przypadki użycia:	12
5.1.2	Diagramy bpmn:	12
5.2	Wymagania funkcjonalne	13
5.3	Wymagania нефункционалне	14
5.4	Diagram pakietów	15
5.5	Diagramy przypadków użycia	15
5.6	Historyjki użytkownika	16
5.6.1	Użytkownik	18
5.6.2	Administrator	19
5.6.3	Dom aukcyjny	20
5.7	Diagramy klas	20
5.8	Diagramy sekwencji	23
5.9	Projekt interfejsu użytkownika	26
5.10	Model danych	35
6	Wprowadzenie do renderowania 3D	37
6.1	WebGPU	37
6.2	WebAssembly	40
6.3	Podsumowanie	40

7	Implementacja komponentu renderującego	41
7.1	Okno aplikacji	41
7.2	WebGPU	43
7.2.1	Adapter	43
7.2.2	Urządzenie	44
7.2.3	Zasoby tworzone przez urządzenie	45
7.3	Pętla główna	48
8	Implementacja wyglądu serwisu aukcyjnego	50
8.1	Koncepcja interfejsu użytkownika	50
8.2	Prezentacja poszczególnych widoków	50
8.2.1	Strona główna	50
8.2.2	Strona szczegółów aukcji	51
8.2.3	System nawigacji	51
8.2.4	Strony logowania i rejestracji	52
9	Implementacja dazy danych oraz logiki biznesowej	54
9.1	Struktura części serwerowej	54
9.2	Połączenie z bazą danych	55
9.3	Rejestracja użytkowników	55
9.4	Logowanie użytkownika	57
9.5	Autentykacja użytkownika	58
9.6	Zarządzanie aukcjami	59
9.6.1	Stworzenie aukcji	59
9.6.2	Kategorie aukcji	60
9.6.3	Licytowanie aukcji	61
9.6.4	Wyświetlanie wszystkich aukcji	63
10	Integracja	65
10.1	Kompilacja do WebAssembly	65
10.1.1	Konfiguracja CMake dla Emscripten	65
10.1.2	Generowane pliki wyjściowe	65
10.2	Architektura integracji w React	66
10.2.1	Wzorzec Provider	66
10.2.2	Hooki dostępu do API	66
10.3	Mechanizm React Portals	67
10.3.1	Zasada działania	67
10.3.2	Zarządzanie slotami	68
10.4	Ładowanie modułu WebAssembly	68
10.4.1	Konfiguracja obiektu Module	68
10.4.2	Asynchroniczne wstrzykiwanie skryptu	69

10.5	Wirtualny system plików	69
10.5.1	Montowanie systemu plików	69
10.6	Komunikacja JavaScript-C++	70
10.6.1	Eksportowane funkcje	70
10.6.2	Przekazywanie elementu canvas	71
10.7	Komponent WebGPUCanvas	71
11	Pomiar wydajności	72
11.1	Metodyka pomiaru	72
11.1.1	Macierz TBN	72
11.1.2	Dane testowe	73
11.1.3	Środowisko testowe	74
11.2	Wyniki pomiarów	74
11.2.1	Analiza skalowania czasowego	74
11.2.2	Podział czasu wykonania	74
11.2.3	Przepustowość operacji ładowania	76
11.2.4	Analiza stabilności pomiarów	76
11.3	Wnioski z analizy wydajności	77
12	Testowanie systemu aukcyjnego	79
12.1	Testy jednostkowe	80
12.2	Testy integracyjne	80
12.3	Testy funkcjonalne	81
13	Wnioski	83

1. Wstęp

Z roku na rok rynek e-commerce dynamicznie się rozwija, przyciągając coraz większą liczbę użytkowników. Według badania „E-commerce w Polsce 2024” szacuje się, że w 2024 roku aż 75% internautów dokonuje zakupów w polskich sklepach internetowych, a 36% korzysta z zagranicznych platform e-commerce [10]. Pandemia Covid-19 znacząco przyspieszyła ten trend - w 2020 roku wartość sprzedaży online w Polsce podwoiła się w porównaniu z rokiem poprzednim [10]. Wraz z rozwojem rynku użytkownicy oczekują coraz bardziej innowacyjnych rozwiązań, które zwiększają ich zaufanie do zakupów online i poprawiają jakość doświadczeń zakupowych.

Jednym z kluczowych wyzwań w handlu internetowym, w szczególności w sektorze aukcji online, jest zapewnienie kupującym możliwości dokładnego zapoznania się ze stanem oferowanego przedmiotu. Tradycyjne platformy aukcyjne, takie jak [desa.pl](#), [allegro.pl](#) czy [the-saleroom.com](#) opierają się głównie na zdjęciach i opisach, które nie zawsze w pełni oddają rzeczywisty stan produktu. Brak możliwości szczegółowego obejrzenia przedmiotu może prowadzić do nieporozumień i obniżać zaufanie kupujących. Rozwiązaniem tego problemu może być zastosowanie nowoczesnych technologii wizualizacji, które umożliwiają interaktywne przedstawienie produktów w trójwymiarowej przestrzeni.

W niniejszej pracy inżynierskiej opisano wykorzystanie technologii WebGPU, użytej do przedstawienia trójwymiarowych obiektów. Obecnie najpowszechniejszym stosowanym standardem do renderowania grafiki 3D w przeglądarkach jest WebGL, na którym opiera się zdecydowana większość istniejących rozwiązań – od prostych wizualizacji produktów w sklepach internetowych po zaawansowane aplikacje typu konfigurator 3D czy przeglądarki modeli w branżach takich jak motoryzacja, meblarstwo i sztuka. Na WebGL bazują popularne biblioteki i silniki, m.in. Three.js, Babylon.js, PlayCanvas czy Potree, które znacznie upraszczają tworzenie złożonych scen 3D.

Aktualnie powstaje próba wprowadzenia nowego standardu WebGPU - następcy WebGL, który oferuje dostęp do nowoczesnych interfejsów programistycznych (Vulkan, Metal, Direct3D 12) pozwalających na komunikację z procesorami graficznymi, znacznie wyższą wydajność, lepszą kontrolę nad pamięcią GPU oraz możliwość wykorzystywania shaderów obliczeniowych (compute shaders) [35]. W grudniu 2025 roku WebGPU jest już stabilnie wspierany w przeglądarkach opartych o silnik Chromium (Chrome, Edge, Opera) oraz w Firefoxie - w sumie w 80.71% przeglądarek [9]. Dzięki temu możliwe staje się osiągnięcie jakości i płynności renderowania dotychczas zarezerwowanej dla aplikacji natywnych, przy jednoczesnym zachowaniu pełnej kompatybilności z ekosystemem webowym. Wykorzystanie WebGPU otwiera nowe możliwości tworzenia fotorealistycznych i wysoce interaktywnych wizualizacji przedmiotów aukcyjnych bezpośrednio w przeglądarce, bez konieczności instalowania dodatkowego oprogramowania [5].

2. Cel i zakres pracy

2.1. Cel pracy

Celem pracy inżynierskiej jest zaprojektowanie i implementacja systemu informatycznego, składającego się z dwóch zintegrowanych części: komponentu renderującego modele trójwymiarowe oraz internetowego serwisu aukcyjnego. Praca zakłada przedstawienie implementacji obu elementów systemu oraz sposobu ich połączenia w jeden działający system informatyczny. Część odpowiedzialna za wizualizację modeli 3D będzie wykorzystywać zasoby procesora graficznego w celu zapewnienia płynnego i efektywnego renderowania. Przekompilowany do formatu WebAssembly kod komponentu graficznego zostanie poprawnie uruchomiony przez nowoczesne silniki przeglądarek, oferując wydajność porównywalną z aplikacją uruchamianą natywnie na komputerze. Realizacja części aukcyjnej obejmuje mechanizmy typowe dla serwisów aukcyjnych: publikowanie ofert i zarządzanie licytacjami z odliczaniem czasu do zakończenia, obsługę ofert w czasie rzeczywistym. Interakcja z trójwymiarowym modelem licytowanego przedmiotu będzie możliwa bezpośrednio z poziomu karty aukcji.

2.2. Zakres pracy

Zakres pracy obejmuje następujące kluczowe elementy:

1. Wprowadzenie do technologii WebGPU i WebAssembly.
 - Omówienie podstawowych koncepcji i architektury WebGPU.
 - Opis technologii WebAssembly i jej zastosowań w kontekście aplikacji webowych.
2. Projekt i implementacja komponentu renderującego 3D.
 - Opis projektowania i implementacji komponentu renderującego.
 - Wyjaśnienie procesu kompilacji kodu napisanego w C++ do WebAssembly.
 - Omówienie potencjalnych trudności.
3. Analiza rynku platform aukcyjnych online.
 - Przeprowadzenie analizy głównych platform aukcyjnych.
 - Ocena aktualnie dostępnych technologii wizualizacji.
 - Identyfikacja potrzeb użytkowników i potencjalnych obszarów poprawy.
4. Projekt i implementacja części klienckiej serwisu aukcyjnego.
 - Zaprojektowanie układu stron.

- Stworzenie komponentów frontendowych.
5. Projekt i implementacja części serwerowej serwisu aukcyjnego.
 - Implementacja bazy danych oraz logiki biznesowej.
 - Implementacja ścieżek API.
 6. Integracja z systemem domu aukcyjnego.
 - Opis sposobu włączenia komponentu renderującego do interfejsu użytkownika.
 7. Ewaluacja i perspektywy rozwoju.
 - Testowanie wydajności i testy serwisu aukcyjnego.
 - Przyszłe zastosowania.
 8. Wnioski i podsumowanie.

3. Analiza rynku

Na rynku istnieje wiele serwisów internetowych, na których użytkownicy mogą licytować i kupować przedmioty w czasie rzeczywistym. Jak okazuje się, nawet na największych platformach, nie ma możliwości podglądu obiektu w trzech wymiarach. Witryny takie jak allegro.pl, amazon.pl czy Erli posiadają prostą funkcjonalność podglądu zdjęć, nie wyróżniając się innowacyjnym rozwiązaniem jakim jest rendering 3D licytowanych obiektów. Ogranicza to zdolność konsumenta do dokładnej oceny produktu, może to prowadzić do wyższej liczby zwrotów zamówień lub niższej satysfakcji klientów. Obrazy 2D zapewniają statyczny widok i nie oddają głębi szczegółów, kątów lub faktur. Prowadzi to do tego, że nie znamy rzeczywistego stanu przedmiotu, przez co potencjalny konsument, może zostać wprowadzony w błąd, podczas zakupu. Sytuacja w której klient otrzyma przedmiot niezgodny z jego oczekiwaniami natychmiastowo obniża zaufanie do platformy. Technologia renderingu 3D pozwala na dokładną inspekcję wybranego przedmiotu, poprzez obracanie obiektem, możliwością przybliżania oraz oddalania się od obiektu i renderingu w bardzo wysokiej jakości, zmniejszając przy tym ryzyko transakcyjne.

3.1. allegro.pl

Jest to największy serwis aukcyjny w Polsce, założony w 1999 roku przez Surf Stop Shop. Badania przeprowadzone przez agencję Minds&Roses potwierdzają, że ponad 33% konsumentów zaczyna właśnie wyszukiwanie produktów na wyżej wymienionym serwisie [43]. Podgląd obiektów na tej stronie jest bardzo prosty, gdyż pozwala tylko na podgląd zdjęć, bez możliwości zaawansowanego podglądu licytowanego obiektu. Interfejs użytkownika jest prosty i bardzo intuicyjny, a funkcjonalność licytowania pozwala wielu użytkownikom licytować te same przedmioty bez żadnych problemów. Na tym portalu, każdy użytkownik może zostać sprzedawcą i wystawić własne przedmioty, nie korzystając z żadnego pośrednika, co ułatwia kontakt pomiędzy konsumentem a sprzedawcą, dzięki czemu każdy problem dotyczący sprzedaży może zostać rozwiązany w prosty sposób.

3.2. amazon.pl

Amazon to jeden z największych serwisów e-commerce na całym świecie. Startował jako księgarnia internetowa, której asortyment systematycznie powiększał się, aż do dzisiejszych rozmiarów. Jediną funkcjonalnością szczegółowego podglądu kupowanego przedmiotu jest powiększenie obrazka, który zapewnił nam sprzedający. Nie mamy możliwości podglądu realnego stanu licytowanego przedmiotu. Sam proces kupowania czy licytowania jest bardzo intuicyjny i szybki, przez co użytkownik nie ma problemów. Amazon ma bardzo dobrze rozbudowaną obsługę zamówień, przez co wszystkie problemy są rozwiązywane szybko i bezproblemowo.

3.3. erli.pl

Założona w 2020 polska firma, która zajmuje się sektorem e-commerce. Odpowiada za sprzedaż przedmiotów w kategorii: meble, budowa i remont, ogród, oświetlenie, narzędzia i wyposażenie domu. Platforma ma prosty i przejrzysty układ strony z licytacją przedmiotu, lecz podgląd kupowanego przedmiotu ogranicza się tylko i wyłącznie do powiększania zdjęć oferowanych przez sprzedawcę. Takie rozwiązanie może nie oddawać dobrze faktur i tekstur przedmiotu, przez co użytkownik może zostać wprowadzony w konsternację podczas zakupu. Wyświetlane zdjęcia są na małej części strony, co utrudnia ogląd licytowanego przedmiotu dla użytkowników, gdyż muszą powiększać stronę, aby dobrze obejrzeć zdjęcia zawarte przez sprzedającego, co sprawia że licytowanie przedmiotu staje się dla użytkownika niepraktyczne.

3.4. Wnioski

Analiza serwisów Allegro, Amazon i Erli wskazuje, że brak możliwości podglądu produktów w technologii 3D ogranicza pełną ocenę przedmiotów przez konsumentów. Może to znacząco wpływać na wyższą liczbę zwrotów oraz obniżonym zaufaniem wśród użytkowników platformy. Wdrożenie renderingu 3D stanowi innowację, daje szansę na poprawę doświadczenia użytkownika, zwiększając jego satysfakcję.

4. Użyte technologie i narzędzia

4.1. Komponent renderujący

- **Język C++** – Wysokopoziomowy język programowania stworzony przez Bjarne Stroustrupa, jako dodatek do języka C. C++ jest szeroko używany w aplikacjach wymagających wysokiej wydajności, takich jak gry komputerowe, silniki gier komputerowych, aplikacje graficzne, aplikacje uczenia maszynowego i wiele innych, gdzie wydajność jest kluczowa. Perfekcyjnie wpasowują się w potrzebę stworzenia graficznego komponentu renderującego obiekty 3D. W przedstawionym projekcie wykorzystywany jest C++23 (ISO/IEC 14882:2024), czyli aktualny otwarty standard języka [18]. Zawiera ona w sobie wiele nowych funkcji, oraz całą zawartość wcześniejszych standardów.
- **WebGPU** – Nowoczesny interfejs programowania aplikacji (API), który umożliwia wydajny dostęp do procesora graficznego (GPU) na różnych platformach [40]. U podstawy działania wykorzystuje systemowe interfejsy: Vulkan, Metal, lub Direct3D 12. Zastępuje starszą technologię WebGL, jako nowy główny standard graficzny dla sieci. WebGPU jest wspierane zarówno w Google Chrome, jak i Firefox. Mozilla używa implementacji w języku Rust o nazwie wgpu [39]. Google natomiast stworzyło Dawn, czyli implementację standardu WebGPU w Chromium za pomocą C++ [38].
- **WebGPU-Cpp** – Wrapper dla WebGPU napisany w C++, ponieważ obie implementacje udostępniają plik nagłówkowy w języku C z definicjami funkcji i struktur. Stworzony został przez użytkownika eliemichel i udostępniany jest na platformie Github [42].
- **WebAssembly** – Otwarty standard, który definiuje przenośny format binarny i odpowiadający mu format tekstowy dla programów komputerowych. Głównym celem jest umożliwienie łatwiejszego tworzenia wydajnych aplikacji w przeglądarkach na stronach internetowych. Jest niezależny od platformy oraz wspiera każdy język programowania. Kod wykonywany jest w wirtualnej maszynie stosowej [37].
- **Emscripten** – Całkowicie otwarty kompilator, który umożliwia kompilację kodu C i C++, lub innego języka używającego LLVM, do formatu WebAssembly. Emcc, czyli frontend kompilatora używa Clang, oraz LLVM. Pozwala on na bezproblemową konwersję praktycznie każdego projektu C i C++ do WebAssembly [11]. Emscripten ma na koncie kilka udanych konwersji takich jak: Unreal Engine 4, Quake 3, czy Doom 3, wszystkie działające w przeglądarce [27].
- **WGSL (WebGPU Shading Language)** – Wysokopoziomowy język shaderów dla WebGPU. Umożliwia on tworzenie shaderów, czyli programów wykonywanych na procesorze graficznym. Został stworzony przez W3C GPU for the Web Community Group, aby zapewnić nowoczesny, bezpieczny i przenośny sposób pisanie shaderów dla WebGPU [41].

- **CMake** – Narzędzie do zarządzania budowaniem kodu źródłowego. Zdejmuje z użytkownika konieczność pisania skomplikowanych plików potrzebnych systemom budowania. Jest szeroko używany w projektach C i C++ [2].
- **Ninja** – Lekki system budowania, skupiający się na szybkości. Jest zaprojektowany, aby pliki wejściowe były generowane przez inne wysokopoziomowe narzędzie. Używany do budowania Google Chrome, czy części systemu Android [25].
- **GLFW** – Napisana w C wieloplatformowa biblioteka do tworzenia okien aplikacji używających OpenGL, OpenGL ES i Vulkan [14].
- **GLM (OpenGL Mathematics)** – Napisana w C++ biblioteka matematyczna przeznaczona dla oprogramowania graficznego opartego na specyfikacjach języka OpenGL Shading Language (GLSL). Biblioteka ta doskonale współpracuje z OpenGL, ale zapewnia kompatybilność z innymi bibliotekami i zestawami SDK innych producentów [15].
- **stb_image** – Biblioteka dla C/C++ umożliwiająca załadowywanie obrazów w różnych formatach, takich jak: JPG, PNG, TGA, BMP, PSD, GIF, HDR, PIC [30].
- **tinyobjloader** – Napisana w C++ biblioteka umożliwiająca załadowywanie modeli 3D w formacie OBJ stworzonym przez Wavefront Technologies [33].
- **Dear ImGui** – Napisana w C++ biblioteka umożliwiająca tworzenie lekkich interfejsów graficznych z minimalnymi zależnościami [7].

4.2. System aukcyjny - Frontend

- **React** – Biblioteka JavaScript do budowy interfejsu użytkownika, odpowiedzialna za interaktywne komponenty strony (nawigacja, karty aukcji, formularze). Umożliwia dynamiczne renderowanie widoków bez przeładowywania całej strony [31].
- **Vite** – Narzędzie do budowania aplikacji webowych. Główną zaletą jest szybkie uruchomienie serwera deweloperskiego oraz natychmiastowe odświeżanie zmian kodu w przeglądarce [36].
- **React Router DOM** – Biblioteka zarządzająca nawigacją i routingiem w aplikacji React. W realizowanym projekcie pozwala na płynne przechodzenie między stronami (główna, aukcje, logowanie) bez odświeżania przeglądarki [28].
- **Tailwind CSS** – Framework CSS wykorzystujący klasy użytkowe do szybkiego stylowania komponentów. Odpowiada za cały wygląd wizualny aplikacji – kolory, layout, animacje, efekty glassmorphism [32].
- **TW Elements** – Komponenty UI oparte na Tailwind CSS. Dostarcza gotowe, interaktywne elementy interfejsu (karty, modale, powiadomienia) [34].

4.3. System aukcyjny - Backend

- **npm** – Node Package Manager to menadżer pakietów dla JavaScript. Jest to narzędzie do zarządzania bibliotekami i zależnościami oraz ich wersjami w projektach pisanych w JavaScript. Dzięki niemu można instalować i odinstalowywać pakiety w projekcie [26].
- **Node.js** – Środowisko uruchomieniowe JavaScript po stronie serwera. Wykonuje logikę biznesową aplikacji, obsługuje requesty HTTP i zarządza danymi użytkowników [21].
- **Express.js** – Minimalny framework webowy dla Node.js. Odpowiada za routing API, obsługę żądań HTTP (GET, POST, PUT, DELETE) i middleware (CORS, autentykacja) [12].
- **MongoDB** – Nierelacyjna baza danych NoSQL przechowująca dokumenty w formacie JSON. Zarządza danymi użytkowników, aukcji i sesji w elastycznej, skalowalnej strukturze [23].
- **Mongoose** – Biblioteka ODM (Object Data Modeling) dla MongoDB. Definiuje schematy danych, walidację i zapewnia wygodny interfejs do operacji bazodanowych [24].
- **JWT (jsonwebtoken)** – Standard tworzenia tokenów uwierzytelniających. Odpowiada za bezpieczne logowanie użytkowników i utrzymywanie sesji bez przechowywania stanu na serwerze [3].
- **bcryptjs** – Biblioteka do hashowania haseł. Zapewnia bezpieczne przechowywanie haseł użytkowników poprzez jednokierunkowe szyfrowanie z solą.
- **CORS** – Middleware umożliwiający komunikację między domenami. Pozwala frontendowi komunikować się z backendem (inny port/domena) zgodnie z polityką bezpieczeństwa przeglądarki.
- **dotenv** – Narzędzie do zarządzania zmiennymi środowiskowymi. Przechowuje wrażliwe dane (klucze API, połączenie z bazą) poza kodem źródłowym w pliku .env.
- **validator** – Biblioteka do walidacji danych wejściowych. Sprawdza poprawność formatów (email, URL, numery telefonu) i zabezpiecza przed nieprawidłowymi danymi.

5. Projekt aplikacji

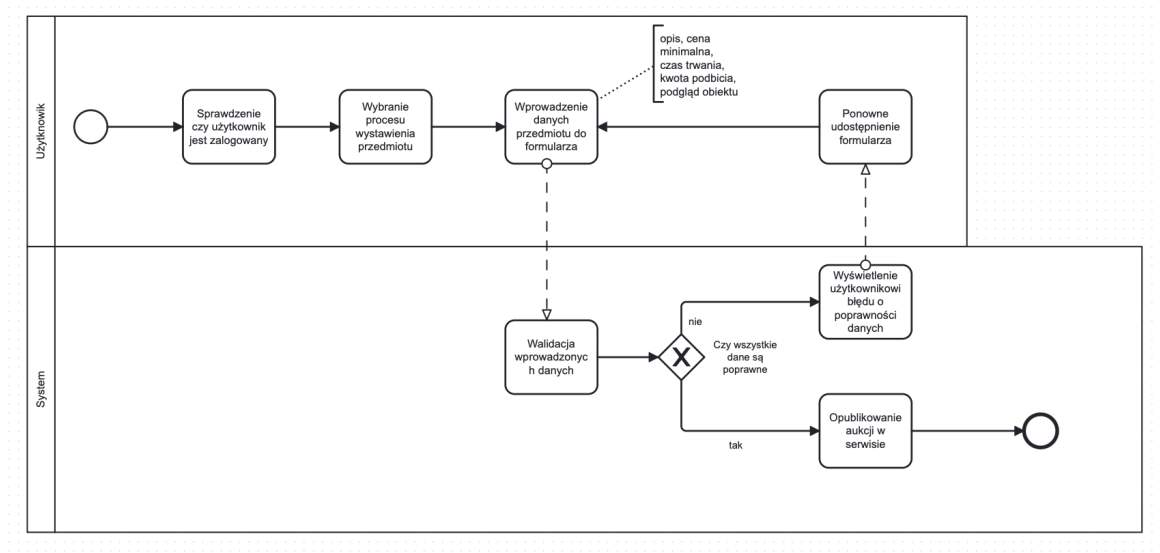
5.1. Udokumentowanie analizy i udokumentowanie rozpoznanych procesów biznesowych

5.1.1. Przypadki użycia:

- Rejestracja nowego użytkownika
- Logowanie do systemu
- Tworzenie aukcji
- Przeglądanie aukcji
- Wizualizacja modelu 3D
- Licytacja
- Moderacja aukcji
- Generowanie raportów i statystyk

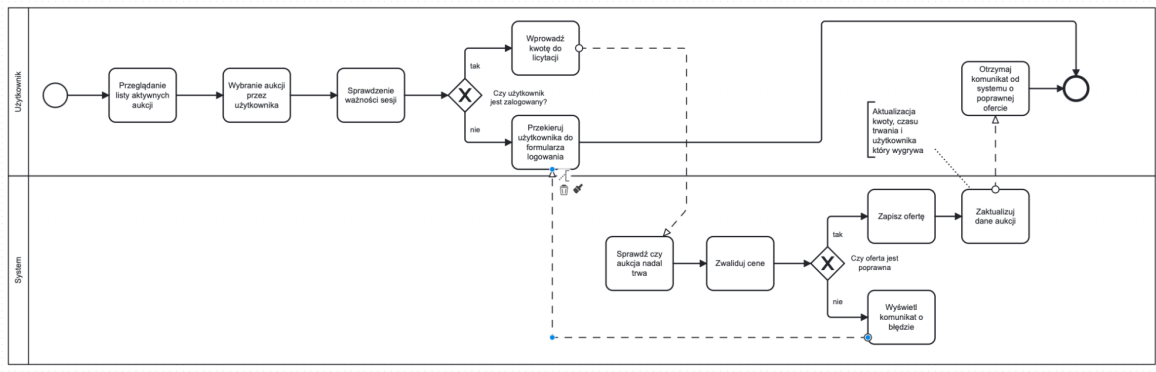
5.1.2. Diagramy bpmn:

Diagramy BPMN (Business Process Model and Notation) służą do modelowania procesów biznesowych. Opisują sekwencje działań i przepływów w procesach danej organizacji.

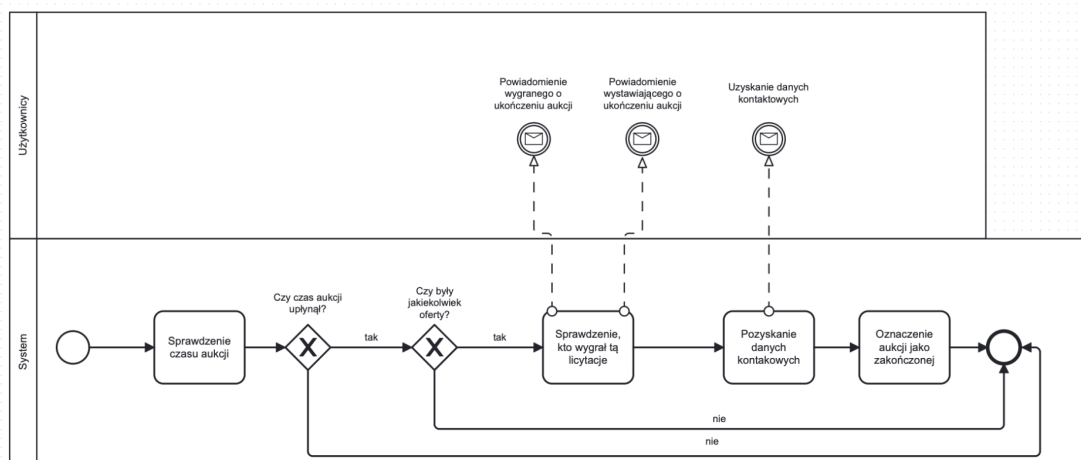


Rysunek 5.1: Tworzenie aukcji

Czy diagramy nie za małe? Można użyć \landscape, ale czy w pracy można to zastosować?



Rysunek 5.2: Proces licytacji



Rysunek 5.3: Kończenie aukcji

5.2. Wymagania funkcjonalne

Wymagania funkcjonalne opisują zakres działania i możliwości systemu.

- Obsługa aukcji
 - System musi umożliwiać tworzenie i zarządzanie aukcjami przez zarejestrowane domy aukcyjne (zakres: kreacja, edycja, usuwanie aukcji).
 - Każda aukcja powinna posiadać ustalony czas rozpoczęcia i zakończenia, automatyczne otwieranie i zamykanie zgodnie z harmonogramem.
 - System musi zbierać i prezentować kluczowe parametry aukcji (status, cena wywoławcza, aktualna najwyższa oferta).
- System licytacji w czasie rzeczywistym
 - Użytkownicy powinni móc składać oferty online w trybie rzeczywistym, z mechanizmem aktualizacji najwyższej kwoty dla wszystkich obserwatorów.
 - Mechanizm obsługi przebijania ofert: każde nowe przebicie powinno być natychmiast komunikowane pozostałym uczestnikom.

- Logika rozstrzygania zwycięzcy oraz rezerwacja przedmiotu po zakończeniu aukcji.
- System wizualizacji modeli 3D
 - Dla każdej pozycji aukcyjnej system zapewnia podgląd modelu 3D przedmiotu, osadzony w interaktywnym viewerze.
 - Viewer 3D powinien obsługiwać podstawowe operacje (obróć, przybliżenie/oddalenie), by użytkownik mógł dokładnie obejrzeć szczegóły oferty.
 - Integracja procesu uploadu i weryfikacji pliku 3D w formatach ustalonych w specyfikacji technicznej.
- Zarządzanie użytkownikami
 - Rejestracja i uwierzytelnianie: tworzenie kont z walidacją danych, logowanie, wylogowanie, procedura resetu hasła.
 - Konta użytkowników z podziałem na role: licytujący, dom aukcyjny, administrator. Każda rola ma inne uprawnienia.
 - Profil użytkownika: edycja danych osobowych, podgląd historii aktywności (aukcje wygrane/przegrane, złożone oferty).
- Interaktywna wyszukiwarka i filtry
 - System wyszukiwania musi obsługiwać pełnotekstowe zapytania, filtrowanie wyników po parametrach (kategoria, przedział cenowy, status aukcji, dom aukcyjny itp.).
 - Sortowanie wyników według różnych kryteriów (cena rosnąco/malejąco, najbliższe zakończenie, najnowsze).
 - Możliwość zapamiętywania i zarządzania własnymi zestawami filtrów (ulubione kategorie, alerty o nowych aukcjach).

5.3. Wymagania niefunkcjonalne

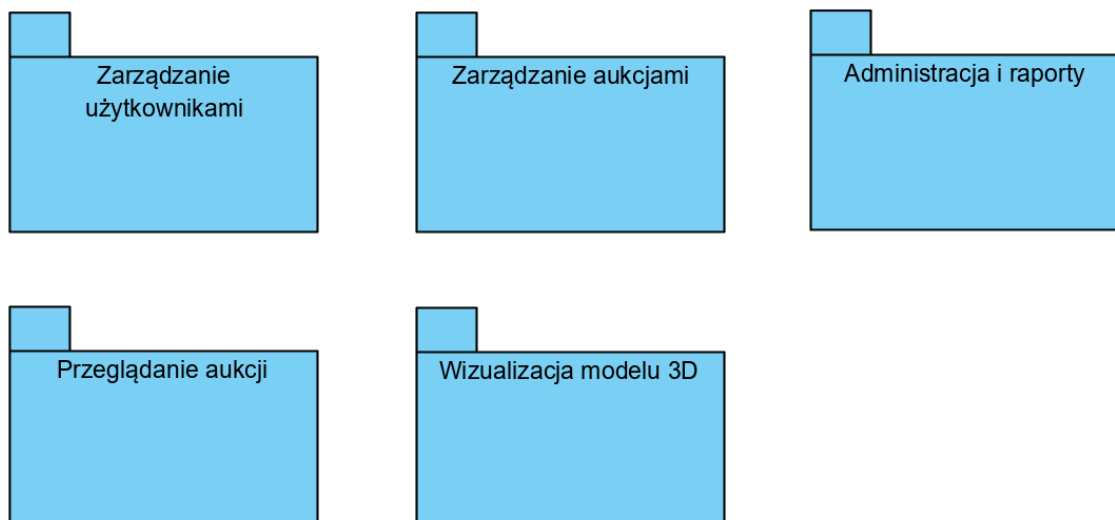
Wymagania niefunkcjonalne opisują działanie systemu od strony wydajności i jego ograniczeń. Nie opisują działania systemu, lecz jego właściwości.

- Wydajność
 - System musi gwarantować płynne renderowanie modeli 3D przy wykorzystaniu WebGPU, zapewniając interaktywność bez opóźnień.
 - Prawidłowa optymalizacja strony pod względem dodawania treści.

- Ogólna skalowalność strony. System powinien pozwalać na obsługiwanie rosnącej liczby użytkowników na aukcji, przy zachowaniu wysokiej wydajności.
- Rozszerzalność i utrzymanie platformy
 - Budowa systemu umożliwiająca dodawanie nowych funkcjonalności bez konieczności przebudowy całego systemu.
 - Przejrzysta dokumentacja kodu i architektury, ułatwiająca przyszłe modyfikacje i integrację z innymi systemami.
- Użyteczność
 - Intuicyjny, przejrzysty i klarowny dla użytkownika interfejs ułatwiający poruszanie się po serwisie zarówno dla strony kupującej jak i sprzedającej.
 - Wsparcie dla nowoczesnych przeglądarek obsługujących WebGPU.
 - Kompatybilność systemu dla wszystkich urządzeń.

5.4. Diagram pakietów

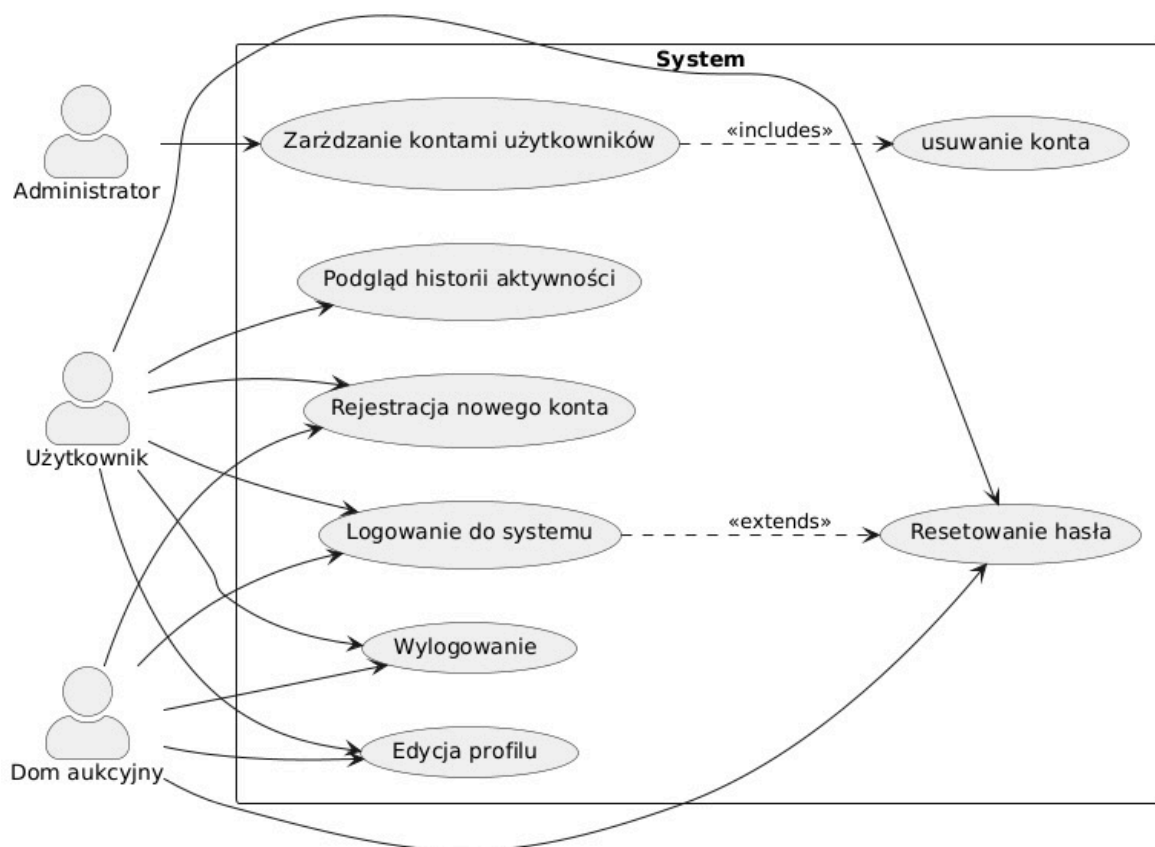
Diagramy pakietów to diagramy UML grupujące przypadki użycia w grupy nazywane pakietami. Diagram ten pomaga organizować przypadki użycia w grupy, poprzez powiązane funkcjonalności. W tym diagramie pakiety są reprezentowane przez foldery, które zawierają przypadki użycia.



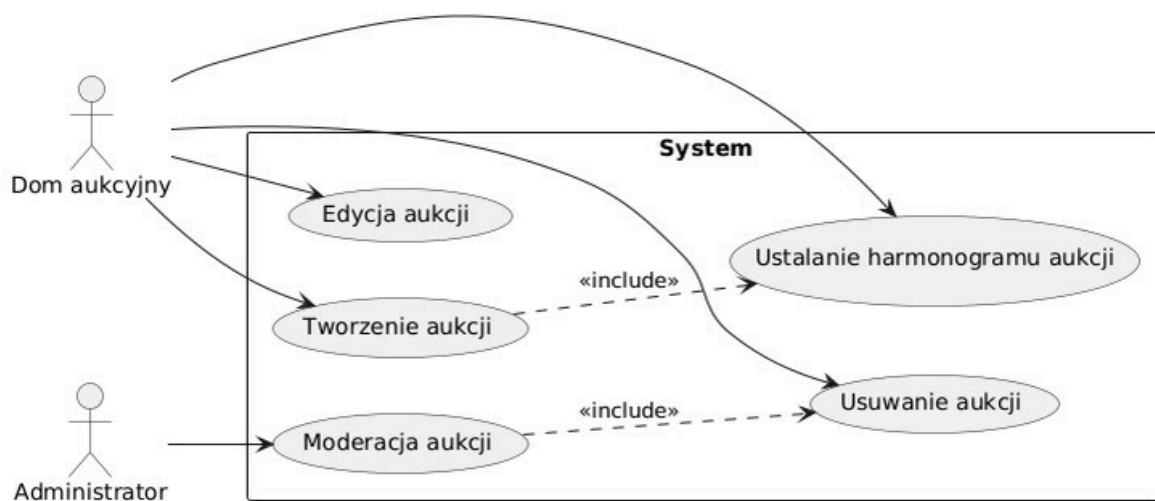
Rysunek 5.4: Diagram pakietów

5.5. Diagramy przypadków użycia

Diagramy przypadków użycia to diagramy UML ukazujące funkcjonalności systemu od strony użytkownika. Diagram pokazuje jacy użytkownicy mogą wykonywać dane czynności w systemie. Ilustrują relacje pomiędzy aktorami, a funkcjonalnościami systemu.



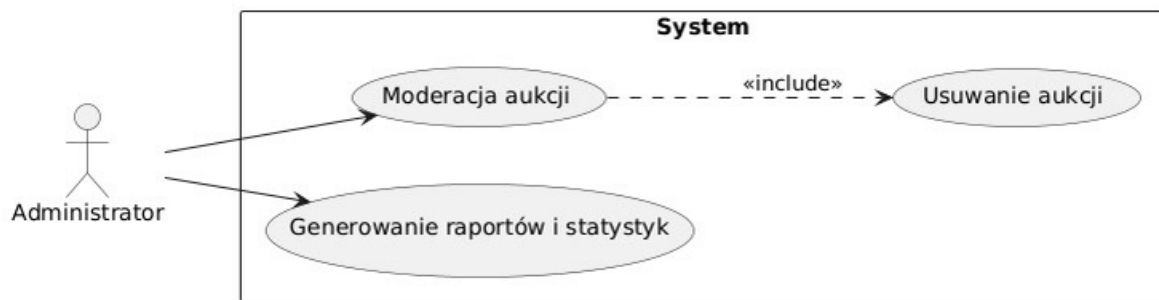
Rysunek 5.5: Diagram przypadków użycia zarządzania użytkownikami



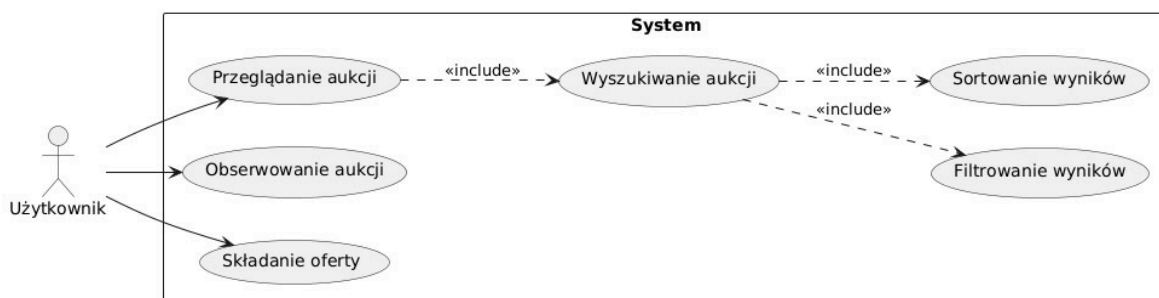
Rysunek 5.6: Diagram przypadków użycia zarządzania aukcjami

5.6. Historyjki użytkownika

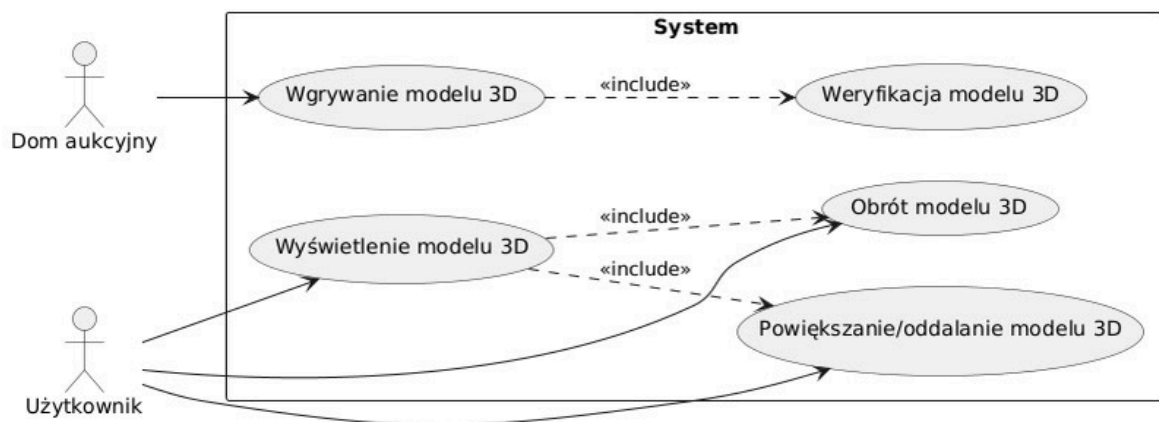
Historyjki użytkownika to opis funkcjonalności systemu, widziany przez użytkownika końcowego. Dzięki historyjkom, wiemy jakich funkcjonalności wymagają poszczególni użytkownicy systemu.



Rysunek 5.7: Diagram przypadków użycia administracji i raportów



Rysunek 5.8: Diagram przypadków użycia przeglądania aukcji



Rysunek 5.9: Diagram przypadków użycia wizualizacji modelu 3D

5.6.1. Użytkownik

- **Jako** użytkownik,
Chciałbym mieć podgląd przedmiotu w 3D, który wybrałem,
Aby móc ocenić stan przedmiotu.
Zakładając, że aukcja zawiera załadowany model 3D,
Jeśli użytkownik kliknie przycisk „Podgląd 3D” na stronie aukcji,
To wtedy interaktywny widok 3D przedmiotu zostanie wyświetlony.
- **Jako** użytkownik,
Chciałbym móc obracać, przybliżać i oddalać przedmiot w podglądzie 3D,
Aby móc dokładniej ocenić stan przedmiotu.
Zakładając, że widoczny jest model 3D w viewerze,
Jeśli użytkownik użyje myszy lub gestów dotykowych,
To wtedy model zareaguje odpowiednio na obrót, przybliżenie i oddalenie.
- **Jako** użytkownik,
Chciałbym w czasie rzeczywistym móc licytować wybrany przedmiot,
Aby dokonać jego zakupu.
Zakładając, że użytkownik jest zalogowany i aukcja jest aktywna,
Jeśli wpisze ofertę wyższą niż obecna i kliknie „Licytuj”,
To wtedy jego oferta stanie się najwyższą i będzie widoczna dla innych w czasie rzeczywistym.
- **Jako** użytkownik,
Chciałbym móc wyszukać przedmiot, który mnie interesuje i wyświetlić 10 przedmiotów, które pasują do wyszukanego przedmiotu,
Aby mieć przedmioty do porównania.
Zakładając, że użytkownik wpisze zapytanie w wyszukiwarce,
Jeśli wyników jest więcej niż 10,
To wtedy system wyświetli pierwsze 10 najbardziej dopasowanych aukcji do zapytania.
- **Jako** użytkownik,
Chciałbym móc zawęzić ilość wyszukanых przedmiotów przez zastosowanie filtrów ceny (filtrowanie według ceny malejąco i rosnąco),
Aby wyrzucić przedmioty, które nie będą w moim budżecie.
Zakładając, że użytkownik określi przedział cenowy lub wybierze sortowanie po ce-

nie,

Jeśli kliknie „Zastosuj filtr”,

To wtedy system pokaże tylko aukcje mieszczące się w wybranym przedziale lub odpowiednio posortowane.

- **Jako** użytkownik,

Chciałbym móc zawęzić ilość wyszukanych przedmiotów przez zastosowanie filtrów czasowych licytacji (od najstarszego do najnowszego),

Aby móc ominąć starsze licytacje.

Zakładając, że użytkownik wybierze sortowanie według daty rozpoczęcia lub zakończenia aukcji,

Jeśli kliknie „Zastosuj”,

To wtedy system posortuje lub przefiltruje aukcje zgodnie z wybranym zakresem czasowym.

- **Jako** użytkownik,

Chciałbym móc dodać przedmiot do sekcji obserwowane,

Aby mieć przedmioty, które mnie interesują w jednej sekcji.

Zakładając, że użytkownik jest zalogowany i przegląda aukcję,

Jeśli kliknie ikonę „Dodaj do obserwowanych”,

To wtedy przedmiot zostanie zapisany w sekcji „Obserwowane” na jego profilu.

- **Jako** użytkownik,

Chciałbym zobaczyć swoją historię zakupów, których dokonałem,

Aby móc szybko wyszukać zakupiony przedmiot.

Zakładając, że użytkownik jest zalogowany,

Jeśli przejdzie do swojego profilu i kliknie „Historia zakupów”,

To wtedy system wyświetli listę wygranych przez niego aukcji.

5.6.2. Administrator

- **Jako** administrator,

Chciałbym móc moderować aukcje na platformie,

Aby usuwać z serwisu nieodpowiednie oferty i zapewnić bezpieczeństwo użytkownikom.

Zakładając, że istnieje aukcja zawierająca niedozwolone treści,

Jeśli administrator wybierze opcję usunięcia tej aukcji,

To wtedy aukcja zostanie trwale usunięta z systemu.

- **Jako** administrator,
Chciałbym generować raporty i statystyki platformy,
Aby analizować aktywność użytkowników i wyniki aukcji.
Zakładając, że system posiada dane o aukcjach i użytkownikach,
Jeśli administrator wybierze zakres dat i kliknie „Generuj raport”,
To wtedy system przedstawi zestawienie z wykresami lub tabelami odzwierciedlającymi statystyki (np. liczbę aukcji, wartość licytacji, liczbę aktywnych użytkowników).

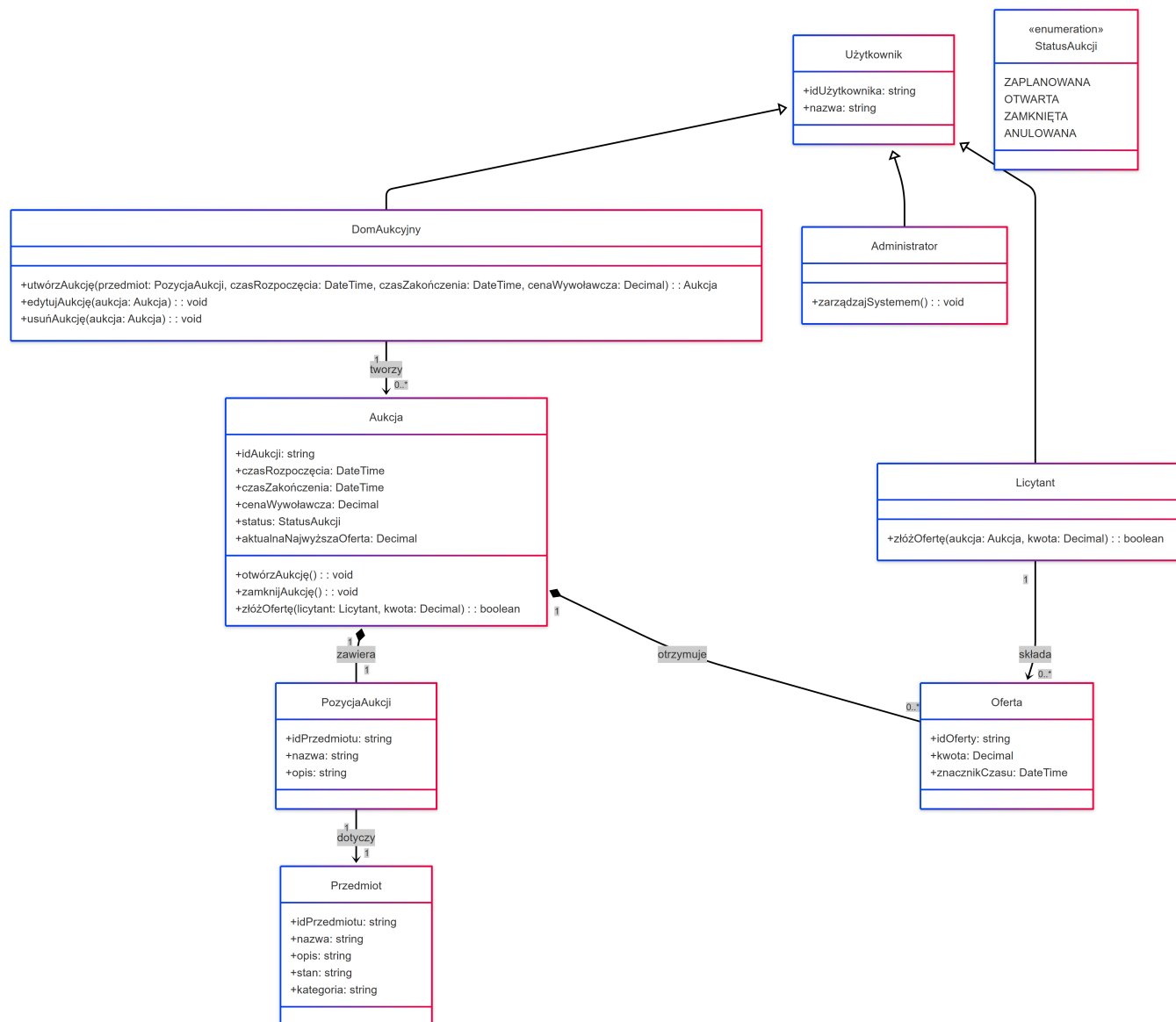
5.6.3. Dom aukcyjny

- **Jako** dom aukcyjny,
Chciałbym zarejestrować konto w serwisie,
Aby móc tworzyć i wystawiać własne aukcje.
Zakładając, że podano prawidłowe dane rejestracyjne (w tym informacje o domu aukcyjnym),
Jeśli formularz rejestracji zostanie poprawnie wypełniony i zatwierdzony,
To wtedy konto domu aukcyjnego zostanie utworzone, a użytkownik uzyska dostęp do panelu aukcyjnego.
- **Jako** dom aukcyjny,
Chciałbym wystawiać nowe aukcje ze szczegółowym opisem i modelami 3D przedmiotów
Aby prezentować oferowane przedmioty w atrakcyjny sposób potencjalnym klientom
Zakładając, że jestem zalogowany jako dom aukcyjny i mam wszystkie dane przedmiotu (opis, zdjęcia, model 3D),
Jeśli wypełnię formularz tworzenia aukcji i potwierdzę jego dodanie,
To wtedy nowa aukcja pojawi się na liście dostępnych aukcji na platformie.

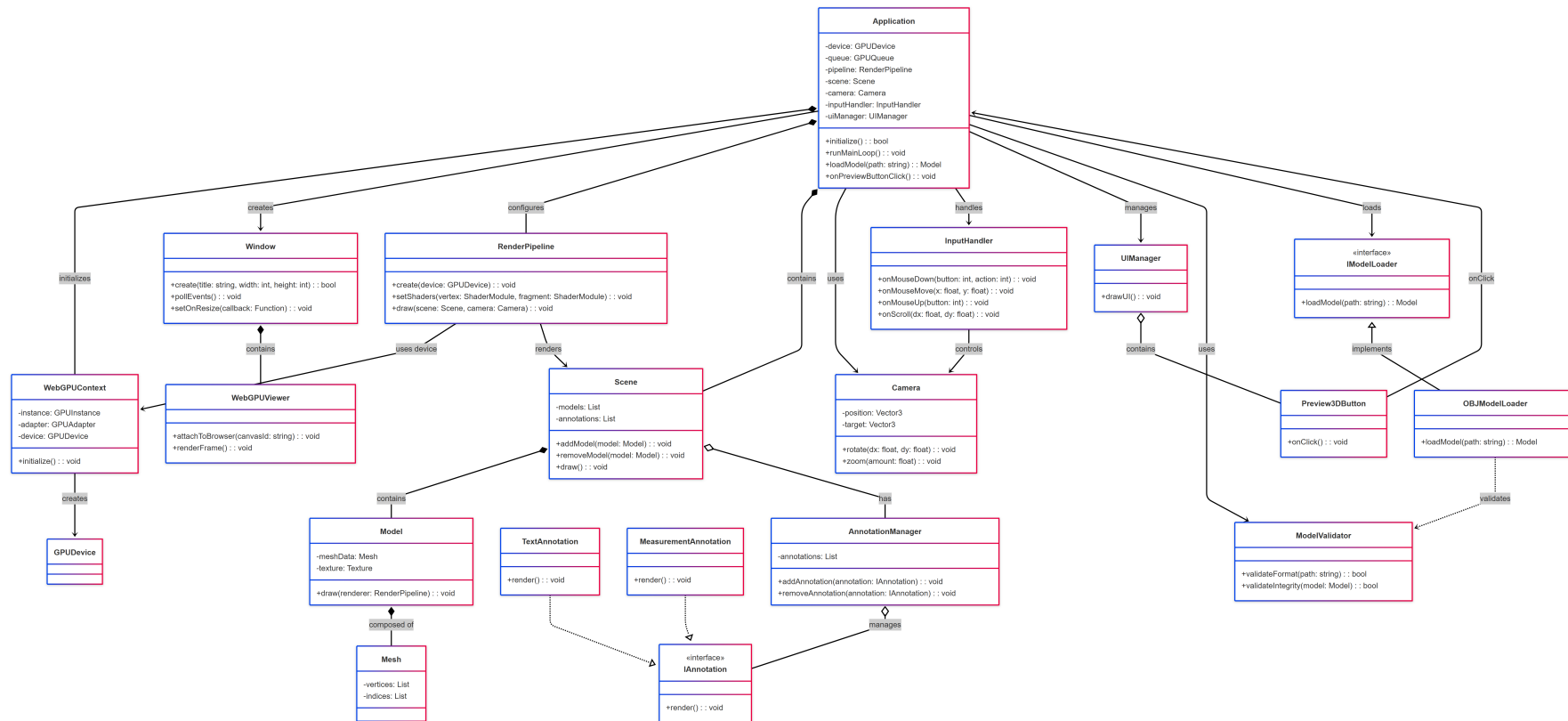
5.7. Diagramy klas

Diagramem klas to typ diagramu strukturalnego UML, który przedstawia strukturę systemu. Pokazuje on klasy, właściwości klas i relacje między klasami. Diagram ten służy jako plan do zaimplementowania bazy danych w systemie.

bez landscape obrazki bardzo nieczytelne



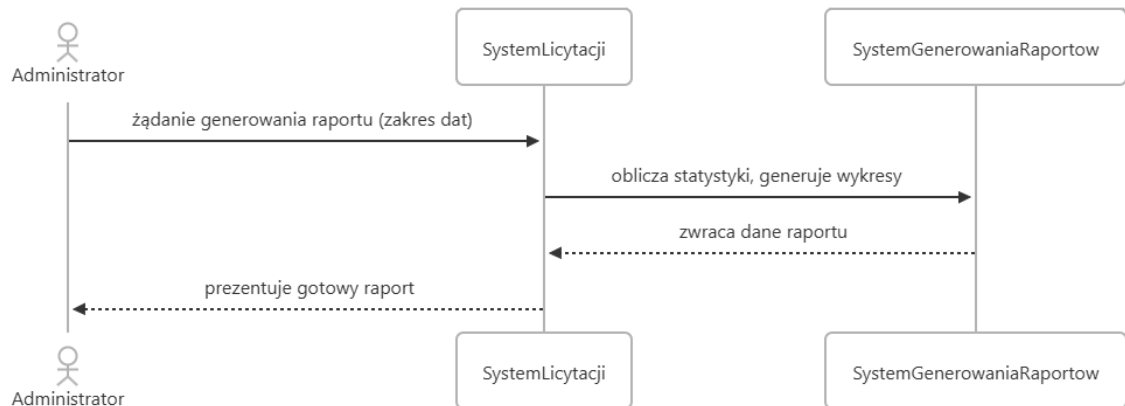
Rysunek 5.10: Diagram klas procesu licytacji



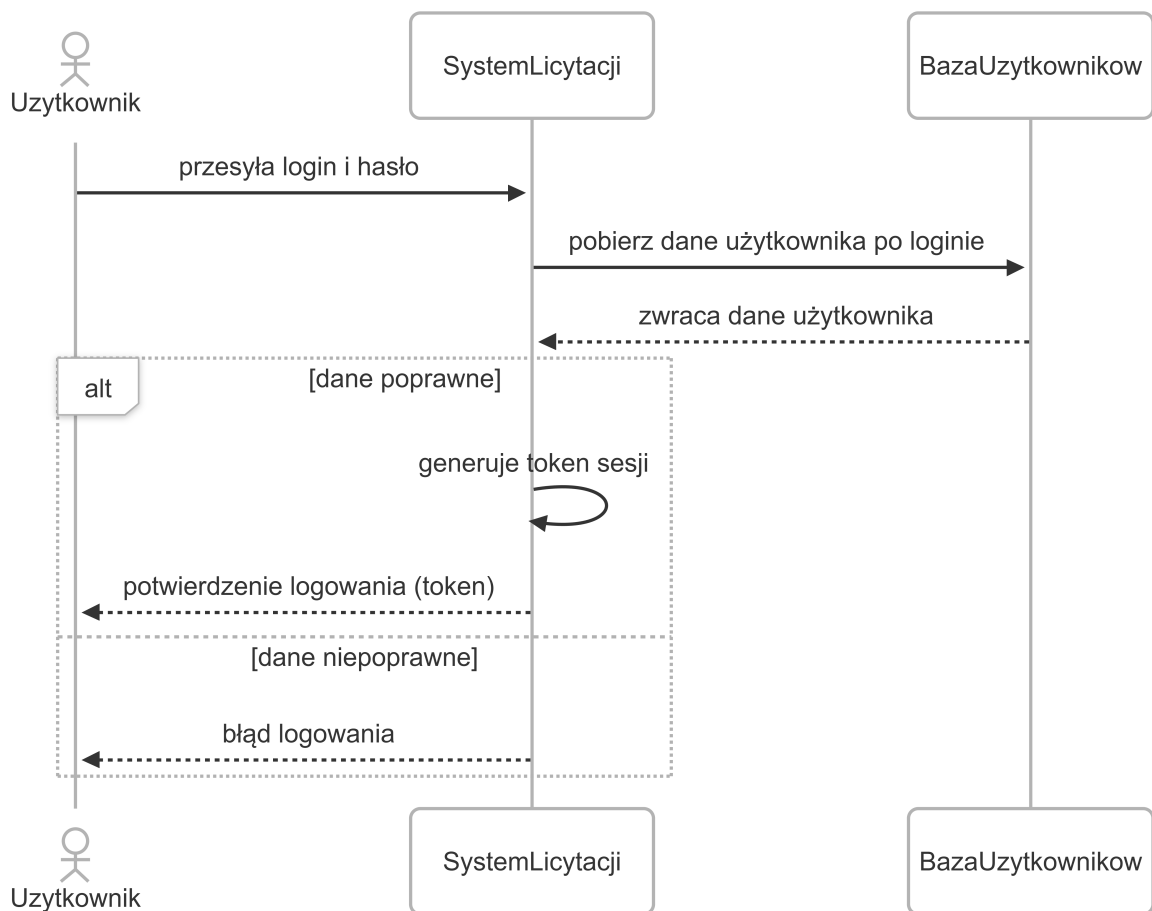
Rysunek 5.11: Diagram klas wizualizacji obiektu 3D

5.8. Diagramy sekwencji

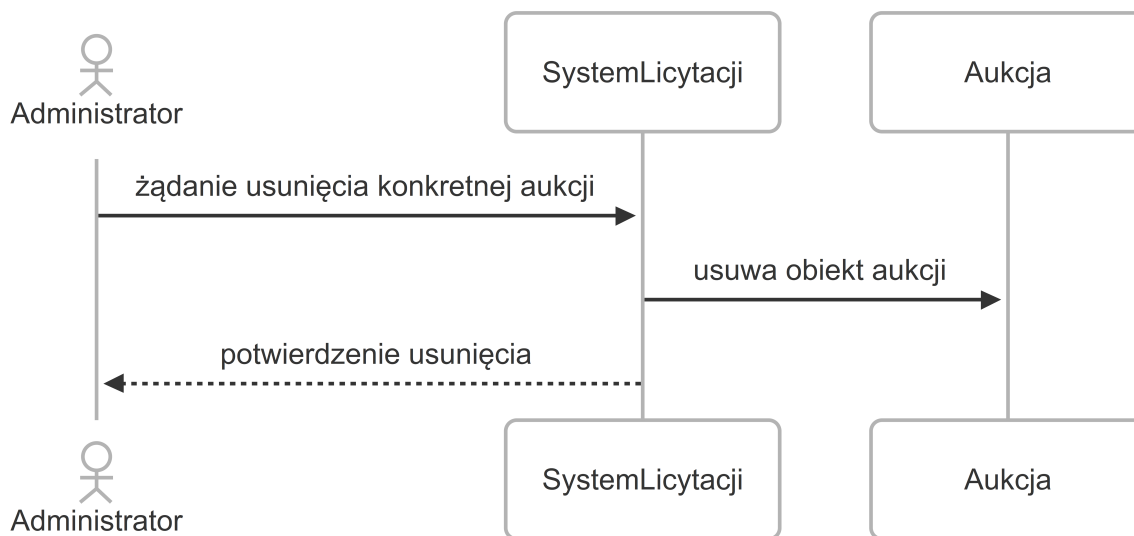
Diagramy sekwencji pokazują współpracę obiektów w systemie podczas działania danego procesu. Obiekty lub aktorzy są przedstawieni na górze diagramu, pionowe linie, zwane liniami życia, odzwierciedlają ich istnienie. Strzałki pomiędzy obiektami, pokazują jak zachodzi komunikacja w systemie pomiędzy poszczególnymi obiektami.



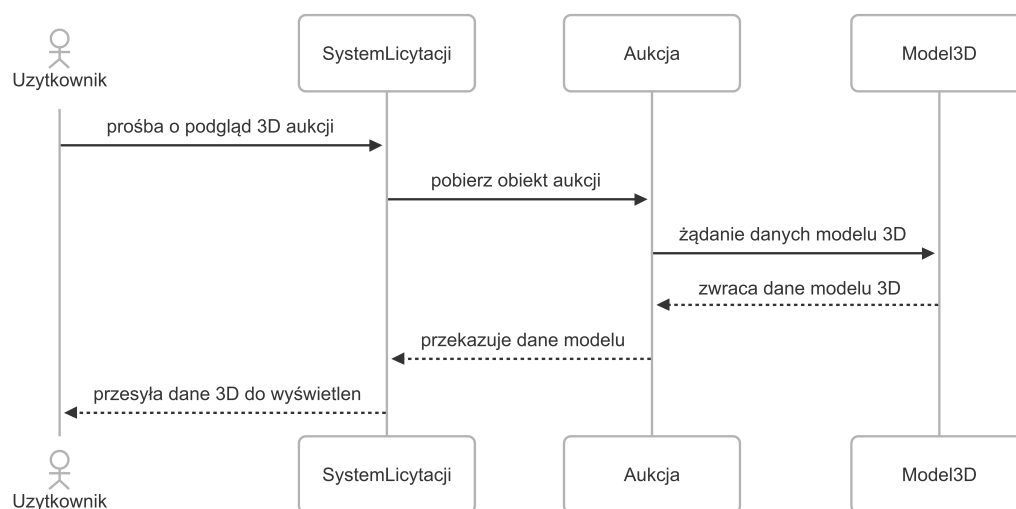
Rysunek 5.12: Diagram sekwencji generowania raportów



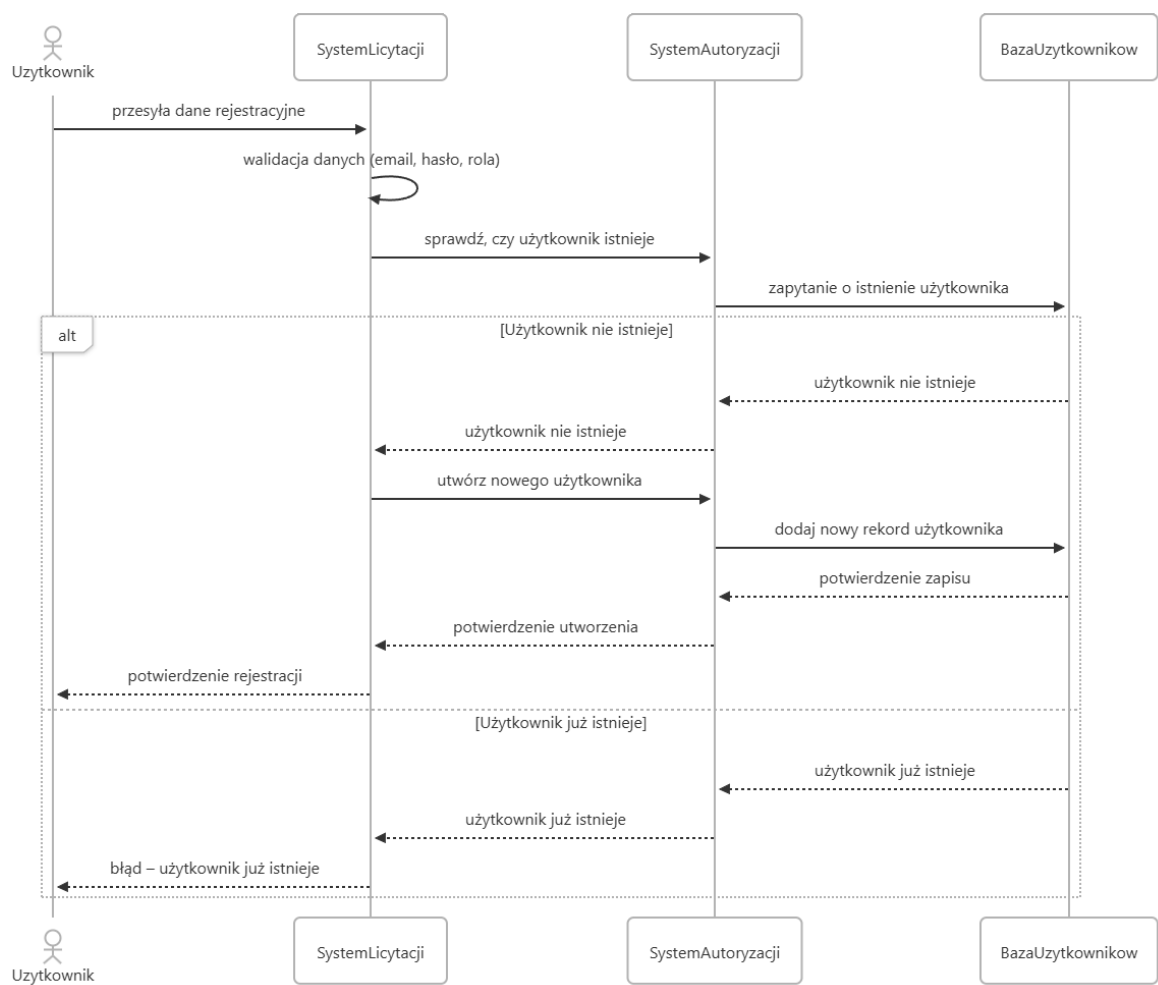
Rysunek 5.13: Diagram sekwencji logowania



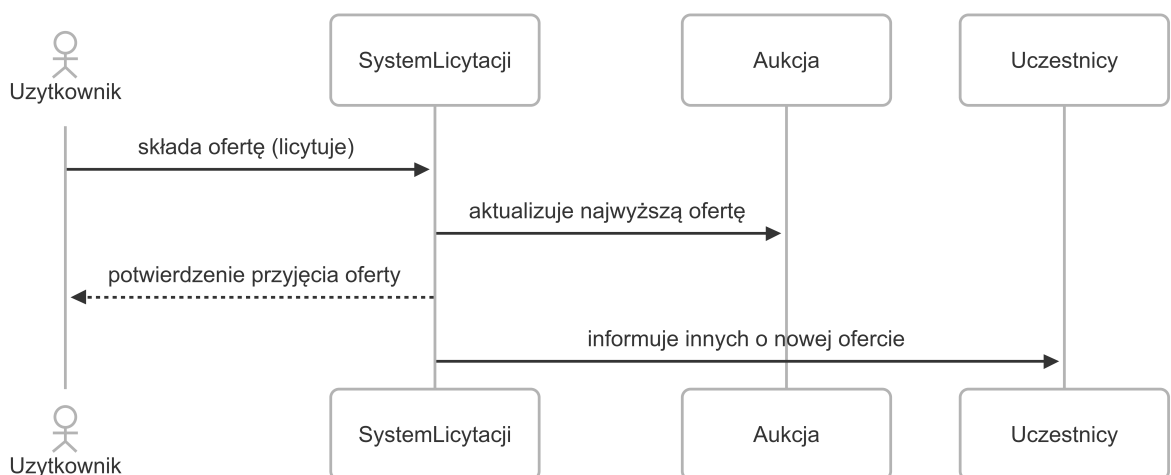
Rysunek 5.14: Diagram sekwencji moderacji aukcji



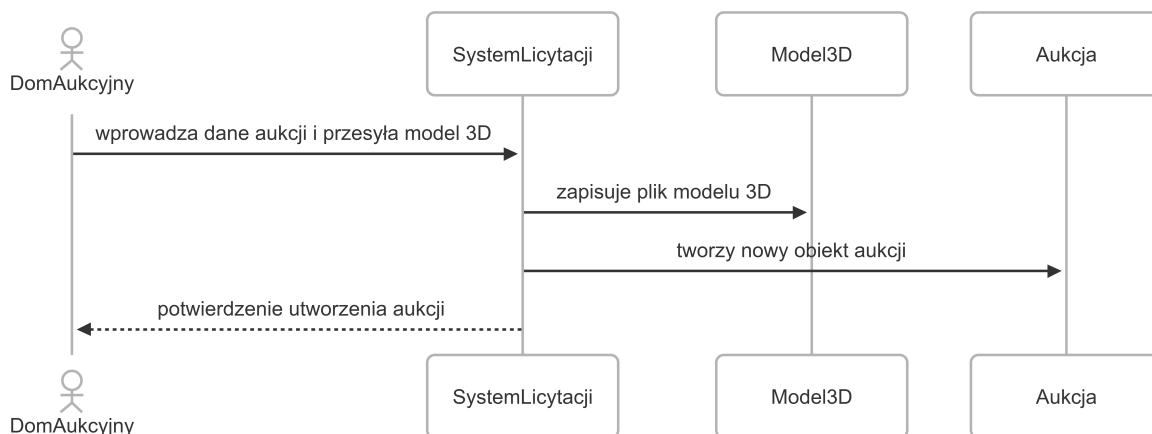
Rysunek 5.15: Diagram sekwencji wizualizacji modelu 3D



Rysunek 5.16: Diagram sekwencji rejestracji



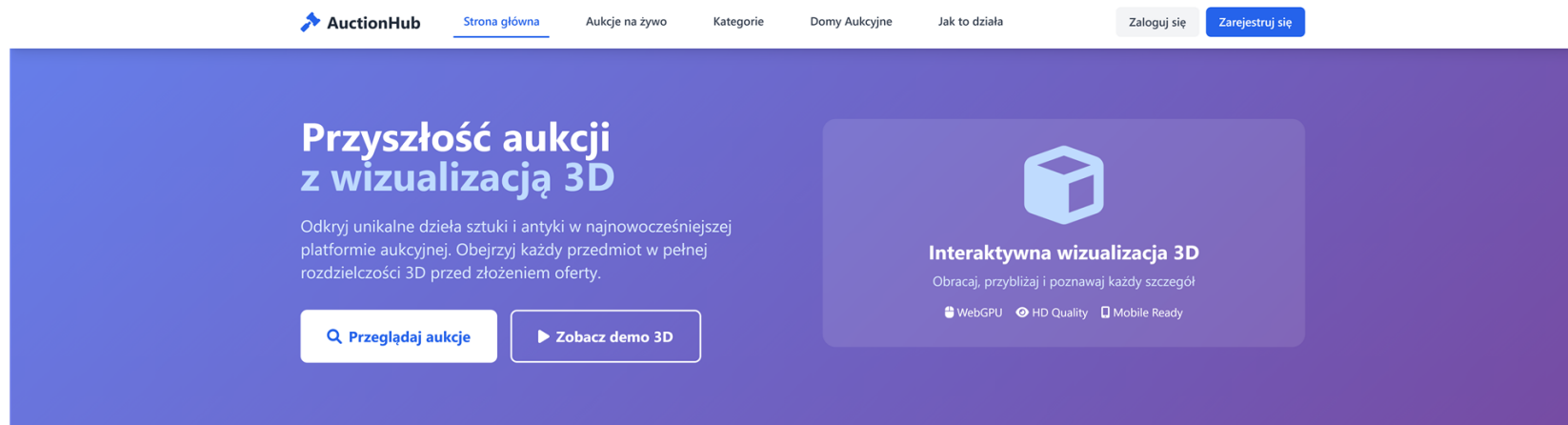
Rysunek 5.17: Diagram sekwencji systemu licytowania



Rysunek 5.18: Diagram sekwencji tworzenia aukcji

5.9. Projekt interfejsu użytkownika

W celu łatwiejszej pracy nad implementacją części klienckiej serwisu aukcyjnego, utworzono widoki, które pokazują ogólny zamysł wyglądu aplikacji. Podczas implementacji interfejsów graficznych, była potrzebna na zmianę niektórych widoków, tak aby interfejs był bardziej przejrzysty i ujednolicony.

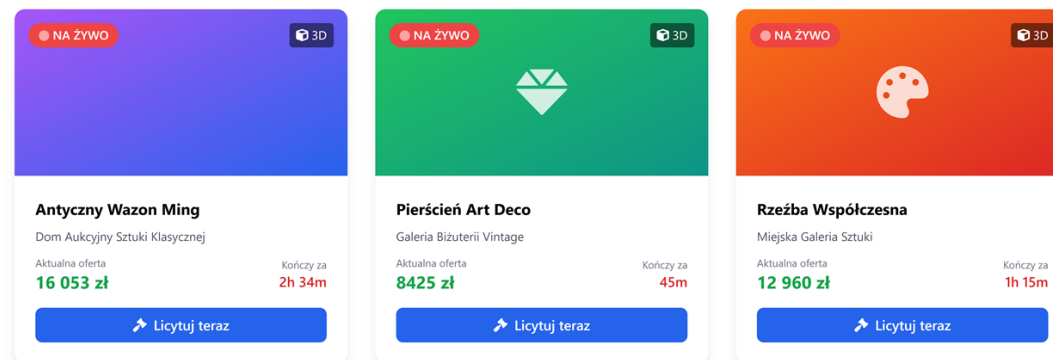


Aukcje na żywo

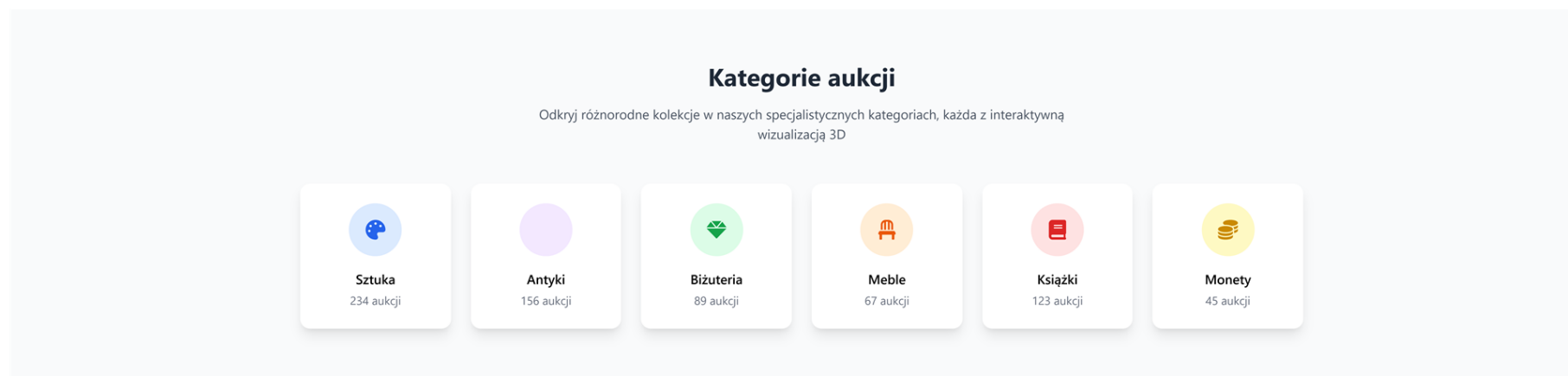
Aktualne licytacje z wizualizacją 3D

10 aukcji na żywo

[Zobacz wszystkie](#)



Rysunek 5.19: Mockup strony głównej - widok 1



Rysunek 5.20: Mockup strony głównej - widok 2

Jak to działa?

Proste kroki do Twojego pierwszego zakupu na aukcji

1

Zarejestruj się

Stwórz bezpłatne konto i zweryfikuj swoją tożsamość w kilka minut

2

Przeglądaj w 3D

Oglądaj przedmioty w pełnej rozdzielczości 3D z każdej strony

3

Licytuj na żywo

Składaj oferty w czasie rzeczywistym podczas trwania aukcji

4

Odbierz przedmiot

Wygrwasz? Odbierz swój przedmiot bezpiecznie i szybko

2456

Zakończonych aukcji

12 849

Aktywnych użytkowników

48

Domów aukcyjnych

125M

Wartość sprzedanych dzieł

Co mówią nasi użytkownicy

Opinie klientów i domów aukcyjnych współpracujących z AuctionHub



Anna Kowalska
Kolekcjonerka sztuki

"Wizualizacja 3D całkowicie zmieniła moje podejście do kupowania na aukcjach. Mogę teraz dokładnie obejrzeć każdy szczegół przed licytacją."



Marek Nowak
Dom Aukcyjny "Klasyka"

"Platforma zwiększyła nasze sprzedaże o 40%. Klienci mają większe zaufanie do przedmiotów, które mogą obejrzeć w 3D."

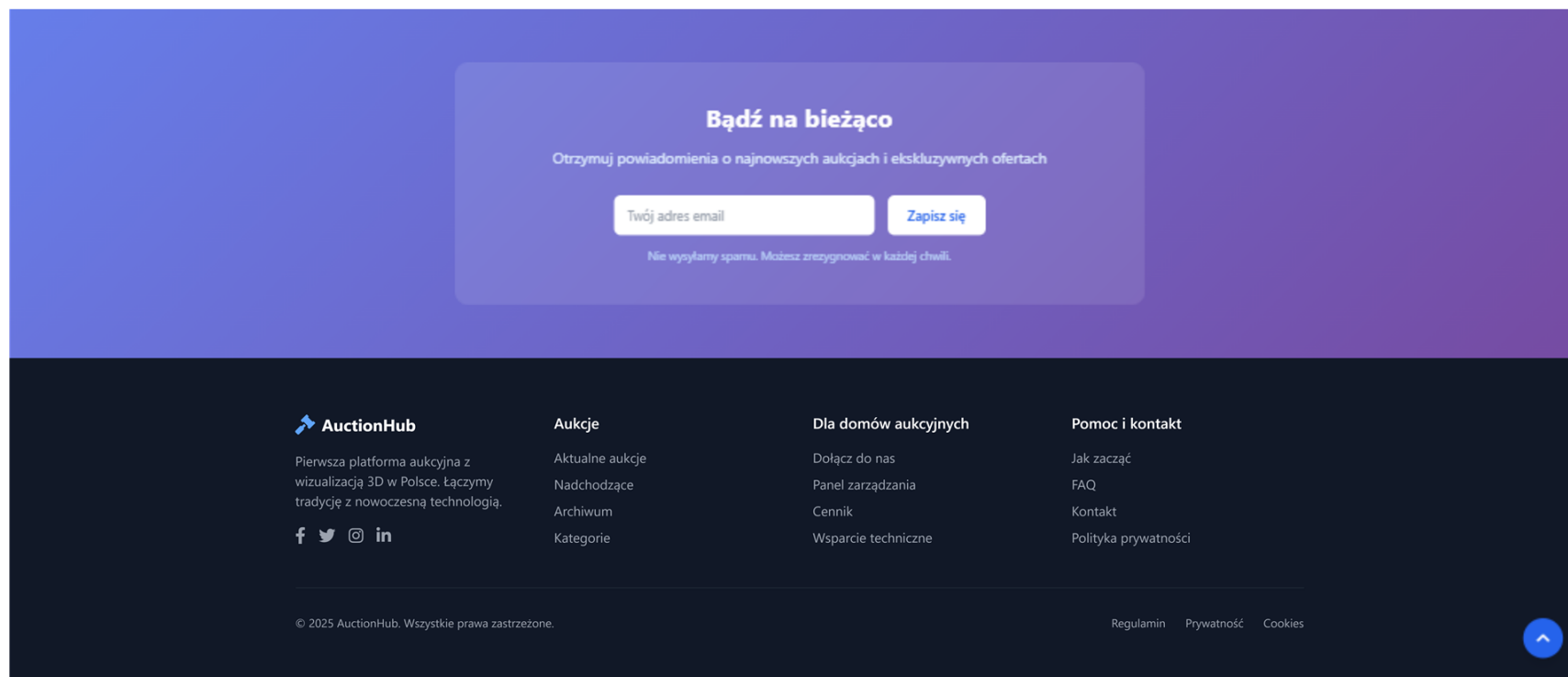


Piotr Wiśniewski
Miłośnik antyków

"Bardzo intuicyjna platforma. Licytowanie w czasie rzeczywistym działa bez zarzutu, a technologia 3D to przyszłość aukcji online."



Rysunek 5.21: Mockup strony głównej - widok 3



Rysunek 5.22: Mockup strony głównej - widok 4

AUKCJA TRWA NA ŻYWO • 247 obserwatorów

Wizualizacja 3D (WebGPU)

Pełny ekran Instrukcje



Model 3D Wazonu Ming

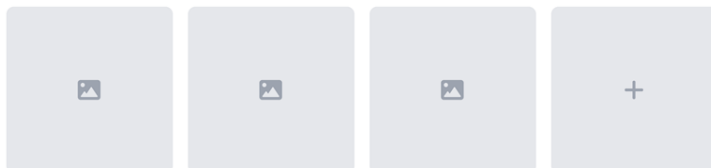
Przeciągnij aby obrócić • Kółko myszy aby przybliżyć

Obrót Zoom Przesuw

Jakość renderowania: Ultra HD

WebGPU aktywne

Dodatkowe zdjęcia



NA ŻYWO



Antyczny Wazon Ming

Dom Aukcyjny Sztuki Klasycznej

Aukcja kończy się za:

02 27 01
godz min sek

Aktualna najwyższa oferta

17 800 zł

Cena wywoławcza: 8,000 zł

Twoja oferta

17900 zł

Minimalna oferta: 17 900 zł (wzrost o 100 zł)

+100 zł

+500 zł

+1000 zł

LICYTUJ TERAZ

Auto-licytacja

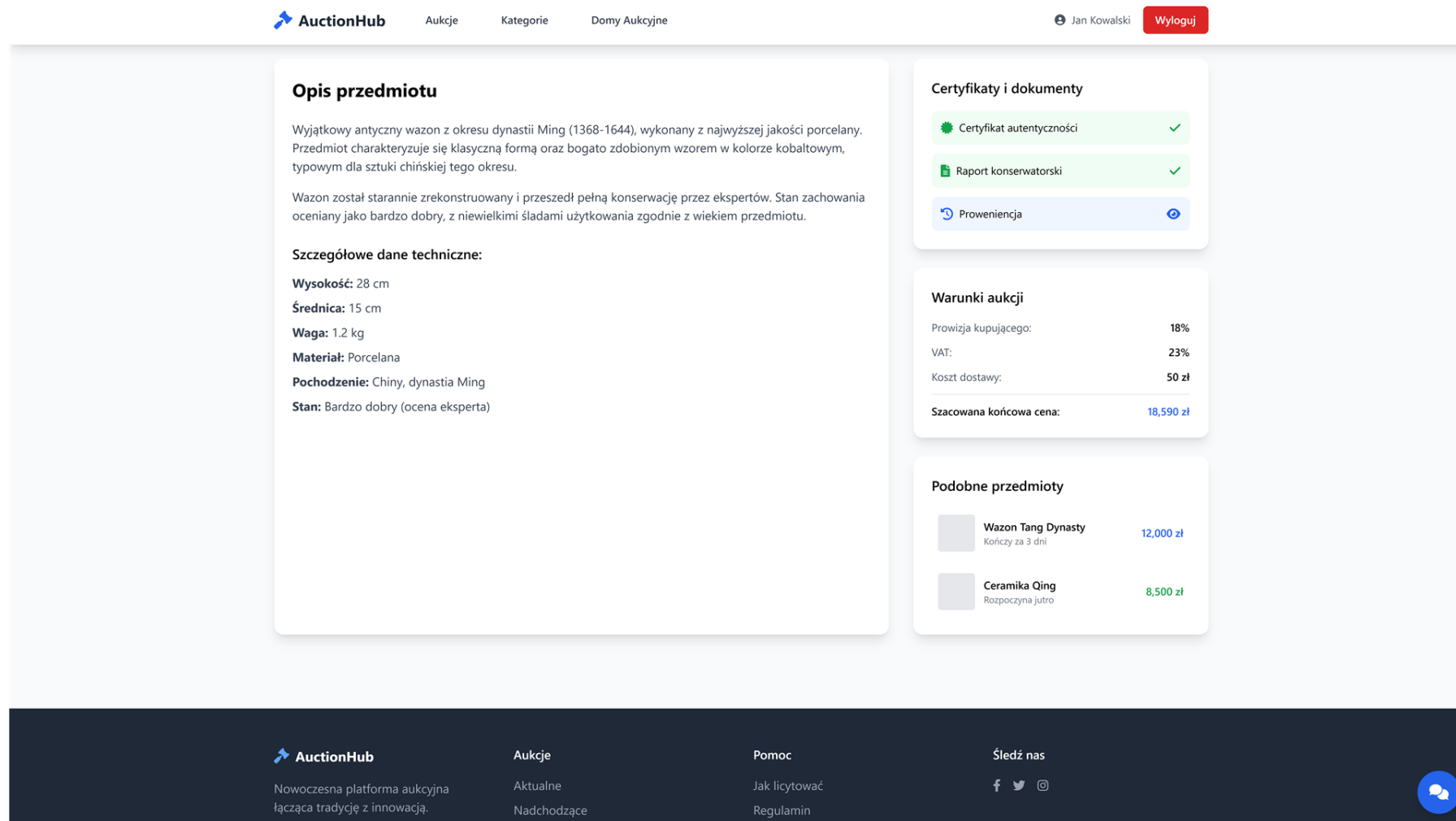
Powiadom

Ostatnie oferty

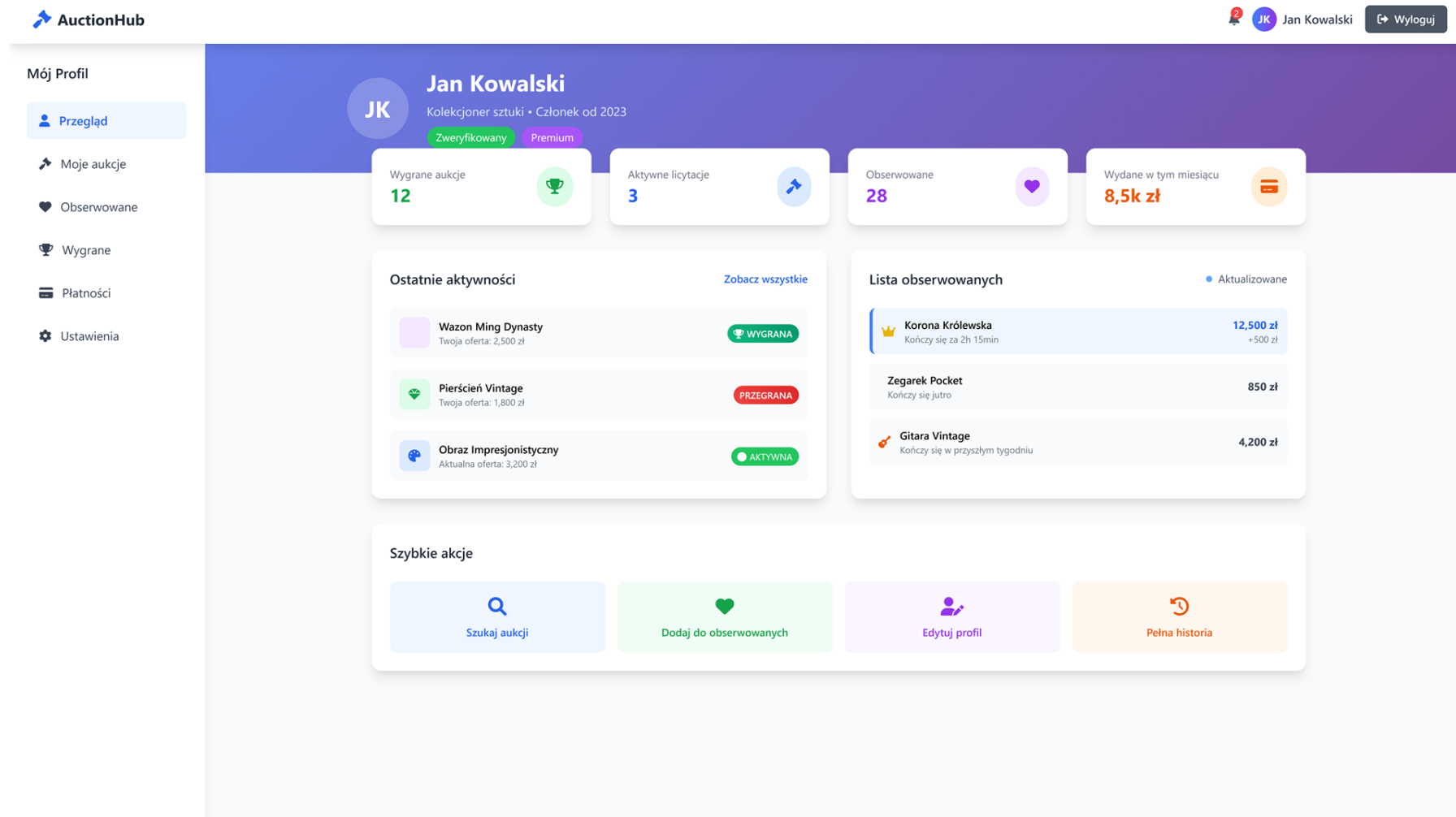
17 800 zł	↑
Użytkownik W***k • teraz	
17 700 zł	↑
Użytkownik P***h • teraz	



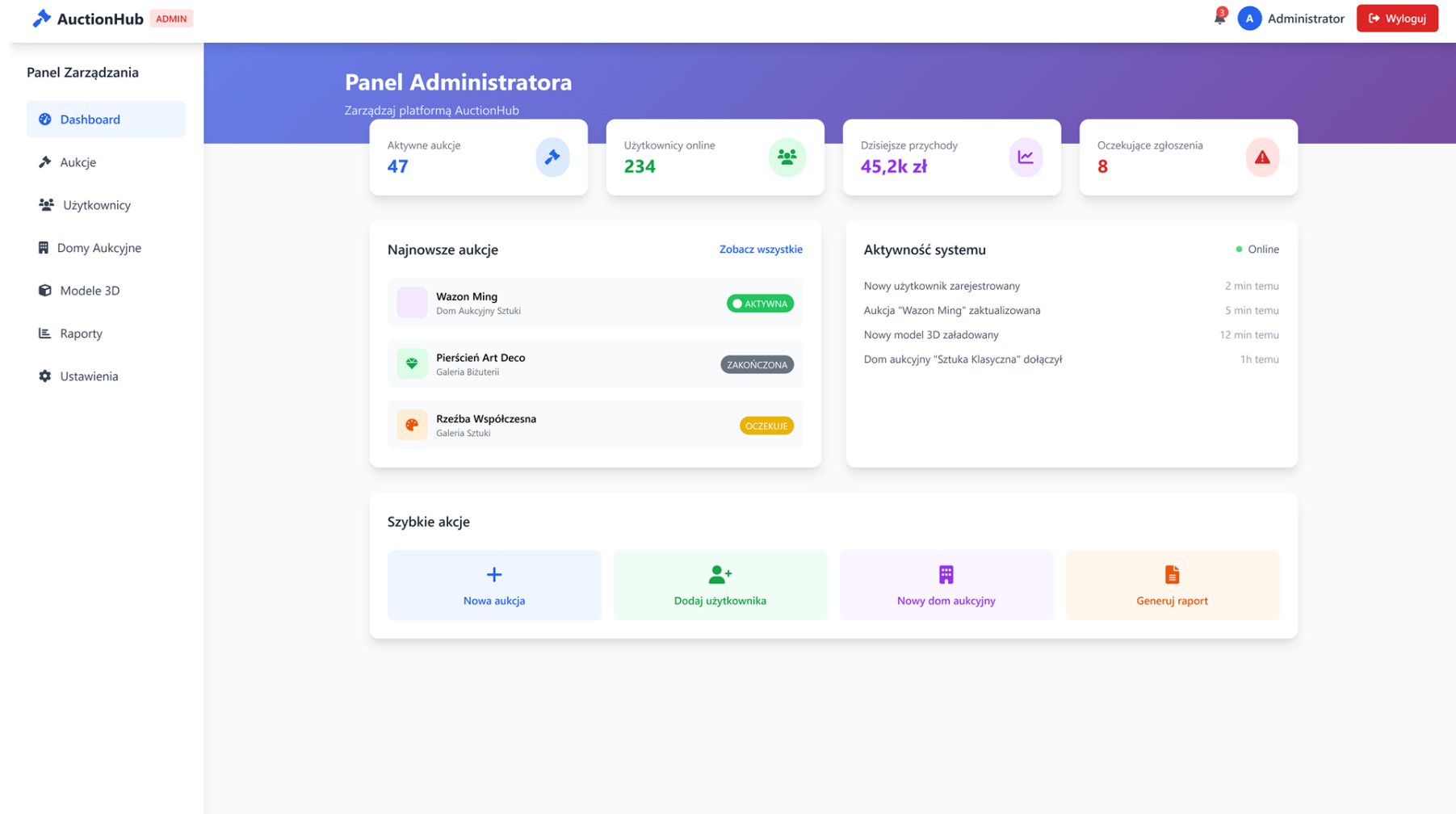
Rysunek 5.23: Mockup widoku aukcji - widok 1



Rysunek 5.24: Mockup widoku aukcji - widok 2



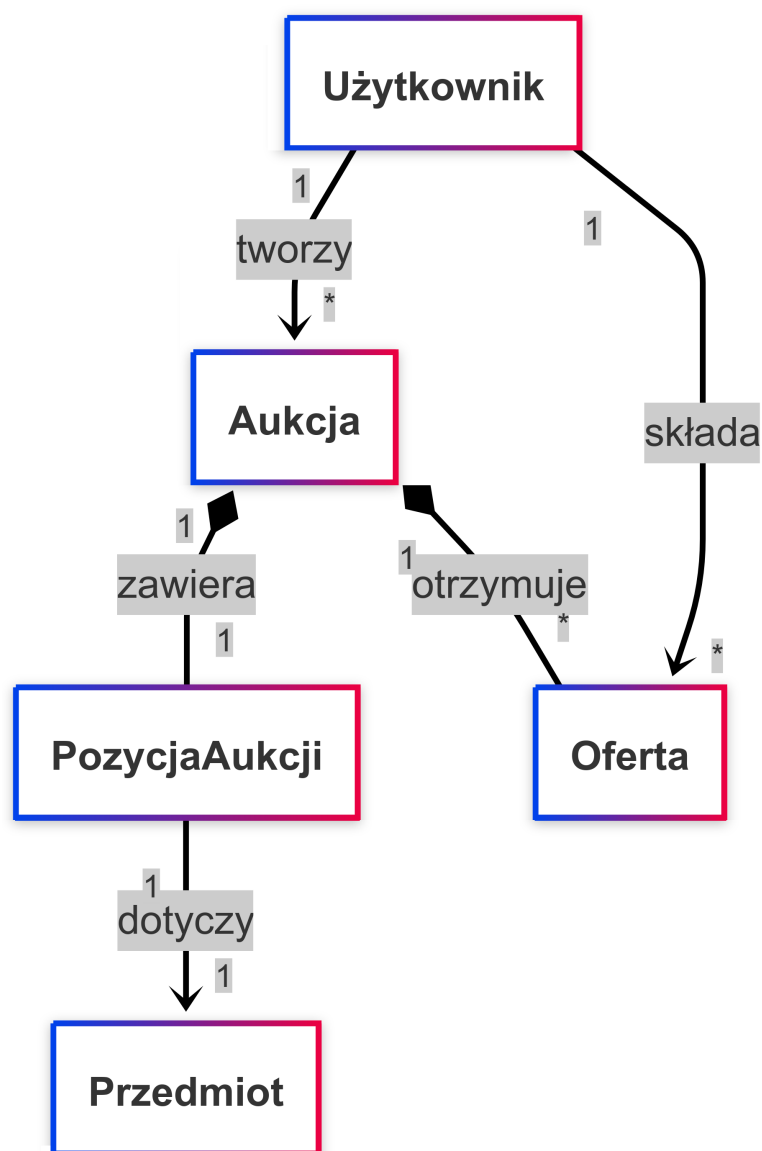
Rysunek 5.25: Mockup widoku panelu użytkownika



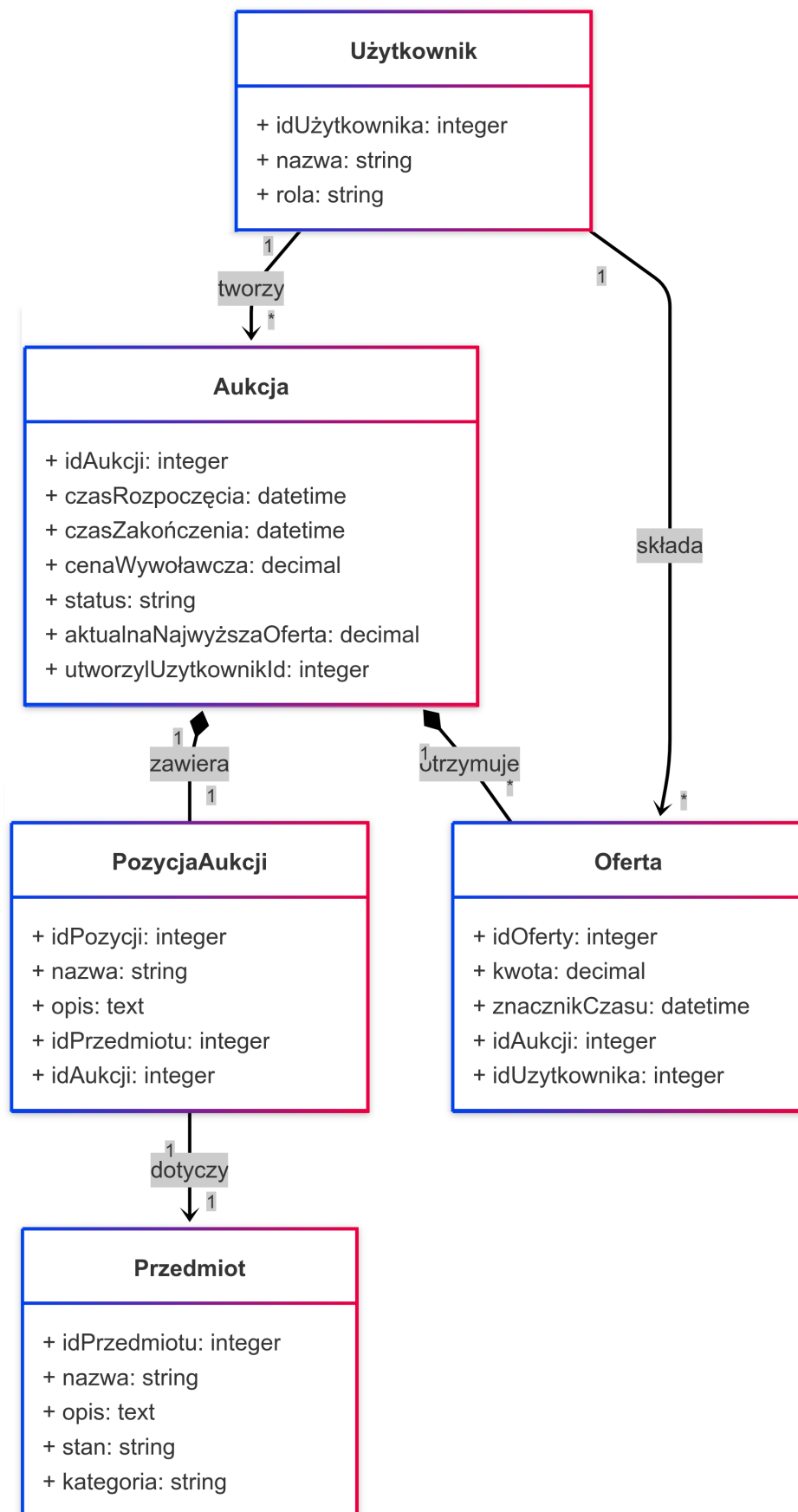
Rysunek 5.26: Mockup widoku panelu administratora

5.10. Model danych

Poniższe diagramy klas prezentują model danych systemu aukcyjnego z kluczowymi encjami (Użytkownik, Aukcja, Przedmiot, PozycjaAukcji, Oferta) oraz ich relacjami; uproszczony diagram pokazuje jedynie nazwy klas i powiązania, a rozszerzony zawiera szczegółowe atrybuty i typy.



Rysunek 5.27: Model danych - uproszczony diagram klas



Rysunek 5.28: Model danych - rozszerzony diagram klas

6. Wprowadzenie do renderowania 3D

Rozpoczynając pracę z zaawansowanymi technikami renderowania 3D w C++, do wyboru są trzy główne biblioteki, które pozwalają na niskopoziomową komunikację z procesorem graficznym: Vulkan, Direct3D 12 oraz Metal. Tylko Vulkan pozwala pisać kod działający na różnych platformach. DirectX jest dostępny wyłącznie na systemie Windows, natomiast Metal jest specyficzny dla ekosystemu Apple. Jednak kod napisany z użyciem tych bibliotek nie może być bezpośrednio uruchomiony w przeglądarce internetowej.

W 2016 roku firma Google zaproponowała stworzenie nowego standardu pozwalającego na komunikację z procesorem graficznym z poziomu przeglądarki internetowej. Miał być to następca WebGL, pozwalający na bardziej efektywne wykorzystanie nowoczesnych kart graficznych. W 2021 roku została opublikowana pierwsza oficjalna specyfikacja pod nazwą WebGPU. Aktualnie na koniec roku 2025 w dalszym ciągu jest to propozycja, czekająca na standaryzację, a nie oficjalny standard W3C. WebGPU jest już wspierane przez większość nowoczesnych przeglądarek internetowych, w tym Google Chrome, Microsoft Edge, Mozilla Firefox oraz Safari. Implementacja WebGPU spaja ze sobą ww. trzy biblioteki, tworząc ujednolicony interfejs programistyczny umożliwiający pisanie przenośnego kodu [29].

Istnieją dwie możliwości korzystania z WebGPU w aplikacjach webowych. Pierwsza z nich to użycie języka programowania JavaScript, które są natywnie obsługiwane przez przeglądarki internetowe. Druga możliwość to skorzystanie z interfejsu programistycznego w języku C udostępnianego przez dwie istniejące implementacje:

- **Dawn** – implementacja stworzona przez firmę Google, przy pomocy języka C++ - używana w przeglądarkach opartych na silniku Chromium. [38]
- **wgpu** – otwarta implementacja, stworzona przy pomocy języka Rust - używana w przeglądarce Mozilla Firefox. [39]

Następnie napisany kod należy skompilować do formatu WebAssembly. Zaletą tego podejścia jest niemalże porównywalna wydajność aplikacji z wersją natywną, gdyż kod WebAssembly jest uruchamiany bezpośrednio przez silnik przeglądarki, a nie interpretowany jak w przypadku języka JavaScript [13, pp. 37–43]. Badania porównawcze wykazują, że WebAssembly oferuje znaczącą przewagę wydajnościową i wykożystywania energii nad JavaScript w aplikacjach wymagających intensywnych obliczeń po stronie klienta [6]. Ponadto istniejący już kod można skompilować także do typowego dla systemu operacyjnego formatu wykonywalnego, co pozwala na uruchomienie aplikacji zarówno w przeglądarce internetowej, jak i natywnie na komputerze.

6.1. WebGPU

Najważniejsze w procesie renderowania jest stworzenie potoku renderowania, dostarczenie do niego danych, oraz przetworzenie ich używając procesora graficznego. Komunikacja z

procesorem graficznym za pomocą WebGPU odbywa się na dwa sposoby. Pierwszy z nich odbywa się za pomocą obiektów reprezentujących wewnętrzne zasoby GPU. Programista ma niewielki wpływ na ich działanie, jedynie jest w stanie wybrać konkretne, wcześniej zdefiniowane ustawienia, które są zawarte w specjalnych strukturach zwanych deskryptorami. uzupełniony deskryptor następnie przekazywany jest jako argument do odpowiedniej funkcji tworzącej zasób. Po zakończeniu działania, zasoby te należy manualnie zwolnić, aby zapobiec wyciekowi pamięci.

```
1 {
2     // ...
3
4     wgpu::SamplerDescriptor sampler_descriptor {};
5     sampler_descriptor.addressModeU = wgpu::AddressMode::Repeat;
6     sampler_descriptor.addressModeV = wgpu::AddressMode::Repeat;
7     sampler_descriptor.addressModeW = wgpu::AddressMode::Repeat;
8     sampler_descriptor.magFilter = wgpu::FilterMode::Linear;
9     sampler_descriptor.minFilter = wgpu::FilterMode::Linear;
10    sampler_descriptor.mipmapFilter =
11        wgpu::MipmapFilterMode::Nearest;
12    sampler_descriptor.lodMinClamp = 0.0F;
13    sampler_descriptor.lodMaxClamp = 8.0F;
14    sampler_descriptor.compare = wgpu::CompareFunction::Undefined;
15    sampler_descriptor.maxAnisotropy = 1U;
16
17    m_sampler = m_device.createSampler(sampler_descriptor);
18    if (m_sampler == nullptr)
19    {
20        return std::unexpected { "Failed to create sampler" };
21    }
22
23    // ...
24 }
```

Listing 6.1: Deskryptor obiektu `WGPUSampler` oraz przekazanie go do funkcji

Drugim sposobem są shadery, czyli programy wykonywane bezpośrednio na procesorze graficznym. Shadery są napisane w specjalnych językach, dla WebGPU jest to WGSL (WebGPU Shading Language). Niezbędne dla prawidłowego działania potoku renderowania są dwa rodzaje: shader wierzchołków (vertex shader) oraz shader fragmentów (fragment shader). Pierwszy z nich przetwarza dane wierzchołków, takie jak pozycja, kolor czy współrzędne tekstury. Shader fragmentów natomiast obliczają kolor i inne atrybuty każdego fragmentu, czyli danych niezbędnych do wygenerowania wartości jednego piksela. Poniżej znajduje się shader fragmentów napisany w języku WGSL. Shader próbkowuje tekstury (koloru

i normalnych), rekonstruuje normalną w przestrzeni świata z użyciem macierzy TBN i miesza ją z normalną geometryczną, oblicza oświetlenie rozproszone i zwierciadlane od czterech światel kierunkowych na podstawie współczynników k_d , k_s i twardości, wykonuje korekcję gamma i zwraca końcowy kolor z kanałem alfa z uniformu.

```
1  @fragment
2  fn fs_main(in: vertex_output) -> @location(0) vec4f {
3      let normal_map_strength = 1.0;
4      let encoded_normal = textureSample(
5          normal_texture, texture_sampler, in.uv).rgb;
6      let local_normal = encoded_normal * 2.0 - 1.0;
7      let local_to_world = mat3x3f(
8          normalize(in.tangent),
9          normalize(in.bitangent),
10         normalize(in.normal),
11     );
12     let world_normal = local_to_world * local_normal;
13     let N = mix(in.normal, world_normal, normal_map_strength);
14     var V = normalize(in.view_direction);
15
16     let base_color = textureSample(
17         base_color_texture, texture_sampler, in.uv).rgb;
18     let kd = u_lighning.kd;
19     let ks = u_lighning.ks;
20     let hardness = u_lighning.hardness;
21
22     var color = vec3f(0.0);
23     for (var i: i32 = 0; i < 4; i++)
24     {
25         let light_color = u_lighning.colors[i].xyz;
26         let L = normalize(u_lighning.directions[i].xyz);
27         let R = reflect(-L, N);
28         let diffuse = max(0.0, dot(L, N)) * light_color;
29         let RoV = max(0.0, dot(R, V));
30         let specular = pow(RoV, hardness);
31         color += base_color * (kd * diffuse) + (ks * specular);
32     }
33
34     let corrected_color = pow(color, vec3f(2.2));
35     return vec4f(corrected_color, u_my_uniforms.color.a);
36 }
```

Listing 6.2: Shadera fragmentów w WGSL

6.2. WebAssembly

W grudniu 2025 roku, 99.24% przeglądarek internetowych wspiera WebAssembly, co czyni go uniwersalnym rozwiązaniem do uruchamiania kodu wymagającego wysokiej wydajności w aplikacjach internetowych [8]. Jednak ten format ten został stworzony jako cel kompilacji, a nie osobny język programowania. W przypadku języka C++ kod źródłowy jest kompilowany do WebAssembly przy pomocy narzędzia Emscripten. Jest to zaawansowane narzędzie, dzięki któremu przeniesiono wiele dużych aplikacji do świata internetowego [27]. Razem z kompilatorem dostarczana także jest specjalna biblioteka, która implementuje standardowe funkcje języków C i C++ w środowisku WebAssembly. Dzięki temu możliwe jest korzystanie z takich funkcji jak alokacja pamięci, operacje na plikach czy obsługa wejścia/wyjścia. Należy jednak pamiętać, że format wyjściowy został stworzony z myślą o bezpieczeństwie i wydajności, dlatego nie wszystkie funkcje języków C i C++ są dostępne lub działają identycznie jak w natywnym środowisku. Maszyna wirtualna, w której wykonywany jest kod nie ma bezpośredniego dostępu do zasobów systemowych, co ogranicza możliwości interakcji z systemem operacyjnym. Emscripten tworzy, więc wirtualny system plików, który symuluje operacje na plikach w pamięci przeglądarki, zachowując przy tym izolację i bezpieczeństwo.

6.3. Podsumowanie

WebGPU w połączeniu z WebAssembly otwiera nowe możliwości dla programistów chcących tworzyć wydajne aplikacje 3D działające bezpośrednio w przeglądarkach internetowych. Dzięki temu możliwe jest tworzenie zaawansowanych wizualizacji, gier oraz interaktywnych aplikacji bez konieczności instalowania dodatkowego oprogramowania. [44] Użycie tych technologii pozwala na pełne wykorzystanie potencjału nowoczesnych kart graficznych, zapewniając jednocześnie szeroką dostępność i łatwość dystrybucji aplikacji.

7. Implementacja komponentu renderującego

Implementację można podzielić w logiczny sposób na trzy główne etapy: inicjalizacja okna, inicjalizacja WebGPU, oraz pętla główna aplikacji. Etapy inicjalizacji muszą być wykonywane w określonej kolejności, dlatego naturalnie układają się w potok. Jednak inaczej niż w typowym potoku zamiast przekazywać między etapami dane, zostanie przekazywana informacja o sukcesie lub błędzie inicjalizacji poszczególnego elementu. W razie wystąpienia błędu, potok zostanie przerwany, a informacja o błędzie wyświetlona. Jest to system podobny w swoim działaniu to wyjątków, jednak dużo bardziej wydajny. Standard C++23 dostarcza do biblioteki standardowej obiekt `std::expected<T1, T2>`, który spełnia wyżej wymienioną funkcjonalność [18]. Użyta zostanie specjalizacja `std::expected<void, std::string>`, gdzie pierwszy parametr szablonu `void` symbolizuje brak danych zwracanych w przypadku sukcesu, a drugi parametr `std::string`, przechowuje komunikat o błędzie. Jak można zauważyć na poniższym listingu, główna funkcja inicjalizująca składa się z dwóch z trzech wcześniej wymienionych etapów, ponieważ pętla główna wykonywana jest po zakończeniu poprawnej inicjalizacji.

```
1 void
2 initialize()
3 {
4     auto result = init_window()
5         .and_then([ this ]() { return init_renderer(); })
6
7     if (!result.has_value())
8     {
9         std::println(
10             std::cerr, "Error occured:\n{}", result.error());
11         std::exit(1);
12     }
13 }
```

Listing 7.1: Funkcja inicjalizująca komponent renderujący

7.1. Okno aplikacji

Pierwszym etapem jest utworzenie okna aplikacji, w którym będzie wyświetlany renderowany obraz. W tym celu wykorzystano bibliotekę GLFW, która umożliwia tworzenie okien oraz obsługę zdarzeń wejściowych w sposób niezależny od platformy. Proces inicjalizacji okna obejmuje inicjalizację biblioteki, oraz stworzenie okna. Wskaźnik na utworzone okno jest zapisywany w klasie głównej aplikacji. Opcjonalnie można też narzucić minimalny rozmiar okna lub wymusić stałą proporcję, co ułatwia utrzymanie spójnej perspektywy w czasie

demonstracji.

```
1 auto
2 init_window() -> std::expected<void, std::string>
3 {
4     if (glfwInit() == GLFW_FALSE)
5     {
6         return std::unexpected { "Failed to initialize GLFW" };
7     }
8     glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
9     glfwWindowHint(GLFW_RESIZABLE, GLFW_TRUE);
10    p_window =
11        glfwCreateWindow(
12            initial_width, initial_height,
13            "INZ", nullptr, nullptr);
14    if (p_window == nullptr)
15    {
16        return std::unexpected { "Failed to create window" };
17    }
18    glfwSetWindowUserPointer(p_window, this);
19    // ...
20 }
```

Listing 7.2: Część kodu odpowiedzialna za inicjalizację okna

Reszta kodu w tej funkcji odpowiada za ustawienie asynchronicznych funkcji obsługi zdarzeń, takie jak zmiana rozmiaru okna czy obsługa myszy.

```
1 auto
2 init_window() -> std::expected<void, std::string>
3 {
4     // ...
5     glfwSetCursorPosCallback(
6         p_window,
7         [](GLFWwindow* window, double x_pos, double y_pos)
8         {
9             if (auto* app = reinterpret_cast<application*>(
10                 glfwGetWindowUserPointer(window)); app != nullptr)
11             {
12                 app->cursor_position_callback(x_pos, y_pos);
13             }
14         });
15 }
```

7.2. WebGPU

Inicjalizacja WebGPU składa się z trzynastu kroków, które muszą być wykonane w określonej kolejności, aby poprawnie skonfigurować środowisko renderujące. Poniżej przedstawiono poszczególne kroki wraz z opisem ich funkcji. W praktyce są one łączone w potok oparty na `std::expected` i operacjach `and_then`, co umożliwia natychmiastowe przerwanie przy pierwszym błędzie oraz czytelne przesłanie komunikatu o niepowodzeniu.

```
1 auto
2 init_renderer() -> std::expected<void, std::string>
3 {
4     return init_adapter()
5         .and_then([ this ] { return init_device(); })
6         .and_then([ this ] { return init_queue(); })
7         .and_then([ this ] { return init_shader_module(); })
8         .and_then([ this ] { return init_depth_texture(); })
9         .and_then([ this ] { return init_bind_group_layout(); })
10        .and_then([ this ] { return init_pipeline(); })
11        .and_then([ this ] { return init_sampler(); })
12        .and_then([ this ] { return init_textures(); })
13        .and_then([ this ] { return init_geometry(); })
14        .and_then([ this ] { return init_uniforms(); })
15        .and_then([ this ] { return init_lighting_uniforms(); })
16        .and_then([ this ] { return init_bind_group(); });
17 }
```

Listing 7.4: Funkcja inicjalizująca WebGPU, stworzona jako potok

7.2.1. Adapter

Adapter to reprezentacja fizycznego lub wirtualnego urządzenia GPU dostępnego w systemie. W tym kroku aplikacja wysyła zapytanie do systemu o dostępne adaptory, a następnie wybiera najbardziej odpowiedni na podstawie określonych kryteriów, takich jak wydajność czy obsługiwane funkcje [40]. Aby móc uzyskać adapter, należy najpierw utworzyć instancję WebGPU (`WGPUInstance`), która stanowi globalny punkt wejścia do interfejsu WebGPU i reprezentuje implementację tego API w systemie. Następnie tworzona jest powierzchnia renderująca (`WGPUSurface`), która stanowi abstrakcję powiązaną z wcześniej zainicjalizowanym oknem GLFW i umożliwia prezentację wyrenderowanych obrazów użytkownikowi. Powierzchnia ta jest przekazywana jako parametr opcji adaptera (`compatibleSurface`), dzięki

czemu wybierany adapter będzie kompatybilny z docelową powierzchnią renderowania. Dopiero po utworzeniu instancji i powierzchni możliwe jest wysłanie zapytania o adapter spełniający określone kryteria kompatybilności.

```
1  auto
2  init_adapter() -> std::expected<void, std::string>
3  {
4      m_instance = wgpuCreateInstance(nullptr);
5      if (m_instance == nullptr)
6      {
7          return std::unexpected { "Failed to initialize WebGPU" };
8      }
9      m_surface = glfwCreateWindowWGPUSurface(m_instance, p_window);
10     if (m_surface == nullptr)
11     {
12         return std::unexpected { "Failed to initialize surface" };
13     }
14     wgpu::RequestAdapterOptions adapter_options {};
15     adapter_options.compatibleSurface = m_surface;
16     m_adapter = m_instance.requestAdapter(adapter_options);
17     if (m_adapter == nullptr)
18     {
19         return std::unexpected { "Failed to create adapter" };
20     }
21     return {};
22 }
```

Listing 7.5: Funkcja inicjalizująca obiekt adapter

7.2.2. Urządzenie

Urządzenie to logiczne przedstawienie wybranego adaptera, które umożliwia tworzenie zasobów procesora graficznego oraz wykonywanie operacji renderujących [40]. W tym kroku aplikacja tworzy obiekt `WGPUDevice`, który będzie używany do komunikacji z procesorem graficznym. Przed jego utworzeniem z adaptera odczytywane są wspierane limity (`WGPULimits`), a następnie definiowany jest zestaw wymaganych limitów przekazywany w deskrypcji urządzenia. Limity te określają oczekiwania aplikacji względem: organizacji geometrii (np. liczby i struktury atrybutów oraz buforów wierzchołków), poprawnych przesunięć dla operacji na pamięci, przepływu danych między etapami shadera (uniformy), a także modelu powiązań zasobów. Obejmują również ograniczenia dotyczące tekstur i samplerów (wymiarów oraz liczebność na etap), tak aby odpowiadały realnym wymaganiom potoku i używanych materiałów. Dzięki zastosowaniu limitów urządzenie jest tworzone tylko wtedy, gdy

implementacja WebGPU gwarantuje parametry wystarczające do bezpiecznego i wydajnego działania. W `WGPUDeviceDescriptor` ustawiane są ponadto etykiety, wskaźnik do wymaganych limitów, opis domyślnej kolejki oraz funkcje zwrotne dla utraty urządzenia i błędów nieprzechwyconych, które służą diagnostyce. Po utworzeniu urządzenia dobierany jest format powierzchni (`BGRA8Unorm`) i wykonywana jest konfiguracja (`configure_surface()`), aby zapewnić poprawne wyświetlanie na utworzonej wcześniej powierzchni.

```
1  auto
2  init_device() -> std::expected<void, std::string>
3  {
4      auto required_limits = get_limits(m_adapter);
5      wgpu::DeviceDescriptor device_descriptor {};
6      // ...
7
8      m_device = m_adapter.requestDevice(device_descriptor);
9      if (m_device == nullptr)
10     {
11         return std::unexpected { "Failed to create device" };
12     }
13     m_surface_format = wgpu::TextureFormat::BGRA8Unorm;
14     configure_surface();
15     return {};
16 }
```

Listing 7.6: Funkcja inicjalizująca obiekt device

7.2.3. Zasoby tworzone przez urządzenie

Po utworzeniu urządzenia (`WGPUDevice`) większość pozostałych zasobów procesora graficznego jest tworzona za jego pomocą. Proces inicjalizacji tych zasobów podąża za wspólnym wzorcem: najpierw tworzony jest obiekt deskryptora, który zawiera szczegółową konfigurację zasobu, następnie wywoływana jest odpowiednia funkcja, która zwraca gotowy zasób. Takie podejście zapewnia spójność i czytelność kodu, a także ułatwia diagnostykę problemów poprzez sprawdzenie, czy zwrócony wskaźnik nie jest pustym wskaźnikiem.

Kolejka poleceń Kolejka poleceń (`WGPUQueue`) jest uzyskiwana bezpośrednio z urządzenia bez potrzeby tworzenia deskryptora [40]. Dzięki kolejce kod aplikacji wykonywany na procesorze komputera może wysyłać polecenia do procesora graficznego w sposób asynchroniczny.

Moduł shaderów Moduł shaderów zawiera skompilowany kod shaderów napisanych w języku WGSL [40]. Shadery definiują sposób przetwarzania wierzchołków (vertex shader) oraz

obliczania koloru pikseli (fragment shader). W implementacji kod shaderu jest ładowany z pliku za pomocą funkcji pomocniczej `load::shader_module()`, która wewnętrznie wykorzystuje `device.createShaderModule()`.

Tekstura głębi Tekstura głębi to specjalny zasób do przechowywania informacji o odległości każdego piksela od kamery w scenie 3D [40]. Jest używana podczas testów głębi w potoku renderowania, aby określić, które fragmenty są widoczne, a które zasłonięte przez bliższe obiekty. W deskrytorze tekstury kluczowe są parametry: format (Depth24Plus - 24-bitowa precyzja głębi), usage (RenderAttachment - użycie jako załącznik renderowania), oraz wymiary odpowiadające wymiarom okna. Po utworzeniu tekstury, tworzony jest także widok tekstury (TextureView) poprzez wywołanie `texture.createView()`, który jest używany w potoku renderowania.

```
1  auto
2  init_depth_texture() -> std::expected<void, std::string>
3  {
4      wgpu::TextureDescriptor depth_texture_descriptor = //...
5      m_depth_texture =
6          m_device.createTexture(depth_texture_descriptor);
7      if (m_depth_texture == nullptr)
8      {
9          return std::unexpected { "Failed to create depth texture" };
10     }
11     auto depth_texture_view_descriptor = //...
12     m_depth_texture_view =
13         m_depth_texture.createView(depth_texture_view_descriptor);
14     if (m_depth_texture_view == nullptr)
15     {
16         return
17             std::unexpected { "Failed to create depth texture view" };
18     }
19     return {};
20 }
```

Listing 7.7: Funkcja inicjalizująca obiekt depth texture

Układ grupy powiązań Układ grupy powiązań definiuje interfejs między zbiorem zasobów a ich dostępnością w poszczególnych etapach shadera [40]. W deskrytorze określone są typy zasobów (bufory uniformów, tekstury, samplery), ich indeksy powiązań oraz widoczność w shaderach wierzchołków i fragmentów. Jest to schemat określający, jakie zasoby będą dostępne dla shaderów, ale nie zawiera jeszcze konkretnych instancji tych zasobów.

Układ potoku i potok renderowania Potok renderowania jest kluczowym elementem procesu renderowania, definiującym wszystkie etapy przetwarzania danych graficznych. Według standardu WebGPU obejmuje on następujące etapy: pobieranie wierzchołków (vertex fetch), shader wierzchołków (vertex shader), składanie prymitywów (primitive assembly), rasteryzacja (rasterization), shader fragmentów (fragment shader), test szablonu i operacje (stencil test and operation), test głębi i zapis (depth test and write), oraz scalanie wyjścia (output merging) [40].

Tworzenie potoku renderowania jest bardziej złożone niż innych zasobów i wymaga wielu struktur konfiguracyjnych. Najpierw tworzony jest układ potoku (WGPUPipelineLayout), który określa ogólną organizację zasobów dostępnych dla shaderów, bazując na wcześniej utworzonym obiekcie WGPUBindGroupLayout. Następnie wypełniany jest deskryptor potoku renderowania (WGPUTemplateDescriptor), który zawiera konfigurację wszystkich wyżej wymienionych etapów. W implementacji poszczególne etapy konfigurowane są przez dedykowane funkcje pomocnicze.

```
1  auto
2  init_pipeline() -> std::expected<void, std::string>
3  {
4      auto vertex_attribs = get_vertex_attributes();
5      auto vertex_buffer_layout =
6          get_vertex_buffer_layout(vertex_attribs);
7      wgpu::RenderPipelineDescriptor pipeline_desc {};
8      // Fill vertex and primitive stage
9      wgpu::FragmentState fragment_state {};
10     wgpu::BlendState blend_state {};
11     wgpu::ColorTargetState color_target {};
12     // Fill fragment stage
13     wgpu::DepthStencilState depth_stencil_state = wgpu::Default;
14     // Fill depth-stencil and multisampling stage
15     auto pipeline_layout_desc = get_pipeline_layout_desc();
16     m_layout =
17         m_device.createPipelineLayout(pipeline_layout_desc);
18     pipeline_desc.layout = m_layout;
19     m_pipeline = m_device.createRenderPipeline(pipeline_desc);
20     if (m_pipeline == nullptr)
21     {
22         return std::unexpected("Failed to create pipeline");
23     }
24     return {};
25 }
```

Listing 7.8: Funkcja inicjalizująca obiekt pipeline

Sampler Sampler koduje informacje dotyczące filtrowania i adresowania tekstur, które są wykorzystywane w shaderach podczas próbkowania (sampling) tekstur [40]. W deskrytorze określone są tryby adresowania dla każdej osi (Repeat - powtarzanie tekstury), filtry powiększania i pomniejszania (Linear - interpolacja liniowa dla gładszego wyglądu), oraz parametry poziomów szczegółowości (LOD).

Bufory Bufor to blok pamięci procesora graficznego, który może przechowywać różnorodne dane wykorzystywane w operacjach graficznych [40]. Wszystkie bufora tworzone są według tego samego wzorca: w deskrytorze określany jest rozmiar bufora, sposób jego użycia (usage) oraz czy ma być mapowany przy tworzeniu. W implementacji używane są trzy rodzaje buforów:

- **Bufor wierzchołków** - przechowuje dane geometrii modeli 3D (pozycje, normalne, współrzędne tekstur). Dane są najpierw ładowane z pliku do pamięci CPU, a następnie przesyłane do bufora procesora graficznego funkcją `queue.writeBuffer()`. Parametr usage: CopyDst | Vertex.
- **Bufor uniformów transformacji** - przechowuje macierze transformacji (model, widok, projekcja), pozycję kamery oraz dodatkowe parametry renderowania. Macierze te realizują podstawowe transformacje geometryczne: translację, rotację i skalowanie obiektu w przestrzeni 3D [20]. Jest aktualizowany w każdej klatce, gdy zmienia się perspektywa kamery lub rotacja modelu. Parametr usage: CopyDst | Uniform.
- **Bufor uniformów oświetlenia** - zawiera informacje o źródłach światła w scenie: kierunku światła, ich kolory oraz parametry modelu oświetlenia (kd, ks, hardness). Parametr usage: CopyDst | Uniform.

Tekstury materiałów Tekstury to wielowymiarowe tablice danych reprezentujące informacje wizualne, takie jak kolory czy mapy normalnych [40]. W implementacji tekstury są ładowane z plików graficznych za pomocą funkcji pomocniczej `load::texture()`, która wewnętrznie używa `device.createTexture()` oraz tworzy odpowiedni widok tekstury.

Grupa powiązań Grupa powiązań łączy konkretne instancje zasobów (bufory, tekstury, samplery) z ich deklaracjami w układzie grupy powiązań [40]. W deskrytorze określany jest wcześniej utworzony obiekt `WGPUBindGroupLayout` oraz tablica wpisów (`WGPUBindGroupEntry`), z których każdy przypisuje konkretny zasób do odpowiedniego indeksu powiązania, co umożliwia shaderom dostęp do tych zasobów podczas renderowania.

7.3. Pętla główna

W pętli głównej wykonywane są ciągłe aktualizacje sceny oraz renderowanie obrazu. Pętla ta działa do momentu zamknięcia okna aplikacji. W każdej iteracji pętli wykonywane

są następujące kroki:

1. Obsługa zdarzeń okna
2. Aktualizacja danych uniformów
3. Zmiana rozmiaru powierzchni renderującej w przypadku zmiany rozmiaru okna
4. Pobranie następnego widoku tekstury powierzchni do renderowania i natychmiastowe przerwanie iteracji jeśli takowego nie ma.
5. Utworzenie enkodera poleceń używanego do nagrywania poleceń procesora graficznego.
6. Stworzenie obiektu render pass. Każda instancja tego obiektu definiuje zestaw zasobów obrazów, zwanych załącznikami, wykorzystywanych podczas renderowania.
7. Ustawienie wcześniej zainicjalizowanych elementów: potoku renderowania, bufora wierzchołków oraz powiązań uniformów.
8. Wykonanie polecenia rysowania.
9. Zakończenie działania obiektu render pass.
10. Sfinalizowanie nagranych poleceń: przesłanie poleceń do kolejki procesora graficznego.

8. Implementacja wyglądu serwisu aukcyjnego

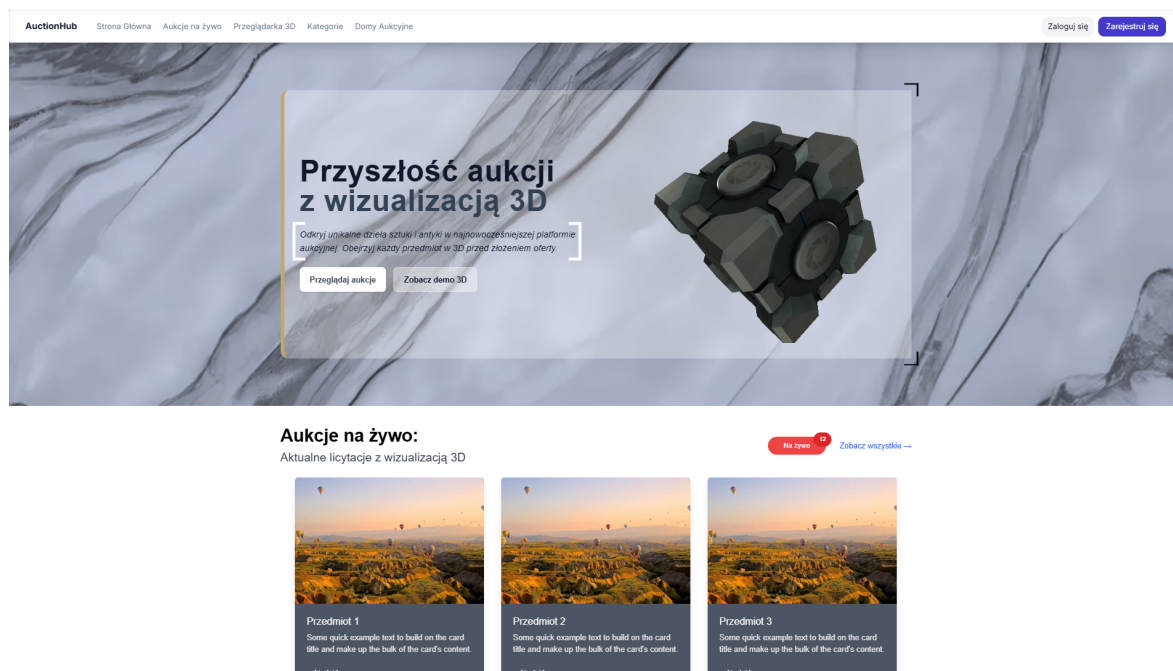
8.1. Koncepcja interfejsu użytkownika

Frontend aplikacji został zaprojektowany z myślą o nowoczesnym, minimalistycznym designie, który kładzie nacisk na prezentację wizualizacji 3D oferowanych obiektów. Interfejs wykorzystuje technologie React oraz Tailwind CSS, co pozwoliło na stworzenie responsywnego i intuicyjnego środowiska aukcyjnego z płynną nawigacją oraz estetycznym układem elementów.

8.2. Prezentacja poszczególnych widoków

8.2.1. Strona główna

Strona główna została zaprojektowana jako wizytówka całej platformy, z charakterystycznym hero section prezentującym główną wartość aplikacji - możliwość przeglądania aukcji z wizualizacją 3D. W centralnej części ekranu znajduje się transparentny panel z nagłówkiem „Przyszłość aukcji z wizualizacją 3D” oraz krótkim opisem: „Odkryj unikalne przedmioty artystyczne w innowacyjny sposób. Zobacz aukcje z wizualizacją 3D zamiast zwykłych zdjęć.”



Rysunek 8.1: Widok strony głównej aplikacji AuctionHub

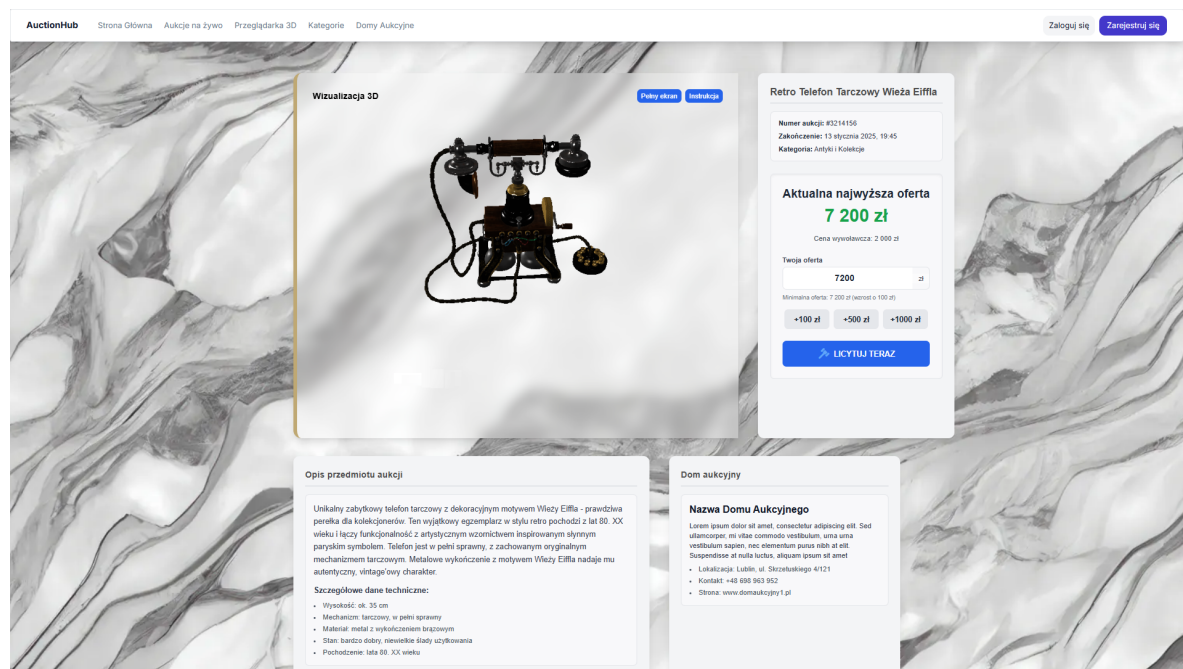
Kluczowym elementem designu jest umieszczenie interaktywnego modelu 3D bezpośrednio na stronie głównej, co natychmiast komunikuje użytkownikowi unikalną funkcjonalność platformy. Panel zawiera dwa przyciski: „Przeglądaj aukcje” oraz „Zobacz demo 3D”, co pozwala użytkownikom od razu zaangażować się w eksplorację platformy.

Poniżej hero section znajduje się sekcja „Aukcje na żywo” z aktualnie trwającymi licytacjami. Wykorzystano tutaj system kart z wizualizacjami 3D aukcji. Każda karta zawiera miniaturę wizualizacji, tytuł aukcji oraz krótki opis. Sekcja posiada również czerwony badge „Na żywo” z licznikiem aktywnych aukcji oraz link „Zobacz wszystkie” dla pełnego dostępu do ofert.

Tło strony wykorzystuje delikatną teksturę marmuru w odcieniach szarości, co nadaje platformie elegancki, luksusowy charakter odpowiedni dla aukcji przedmiotów artystycznych.

8.2.2. Strona szczegółów aukcji

Widok szczegółowy aukcji dzieli ekran na dwie główne sekcje. Po lewej stronie znajduje się duży panel z wizualizacją 3D obiektu. Prawa kolumna zawiera pełne informacje o aukcji. Pod panelem wizualizacji znajdują się dwie sekcje informacyjne.



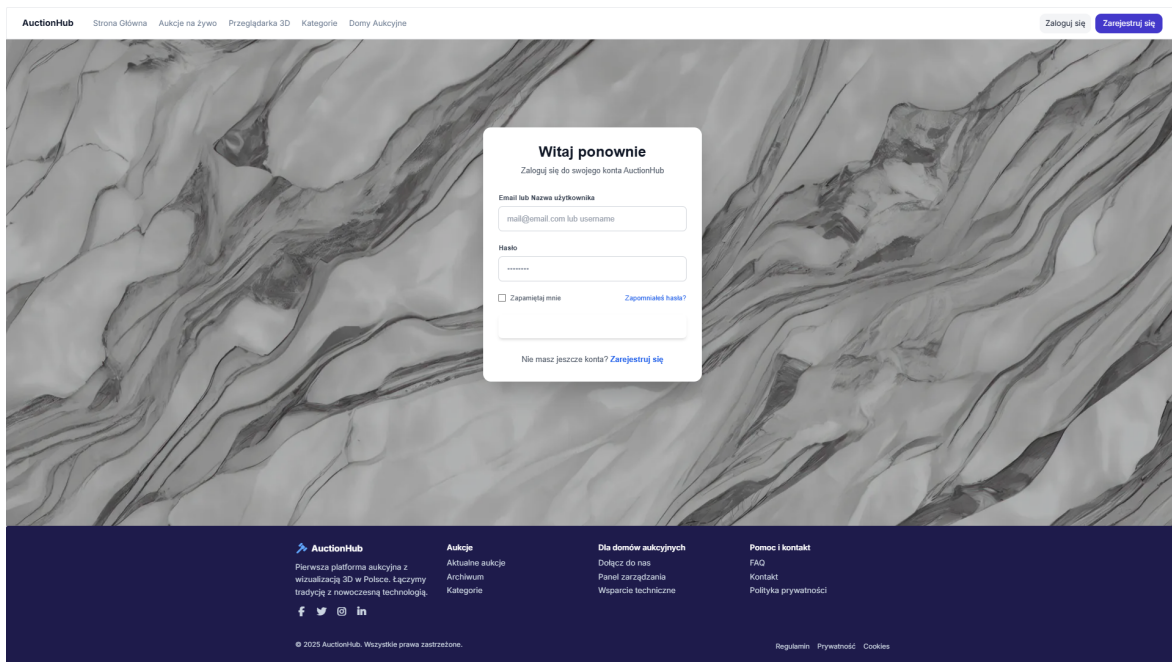
Rysunek 8.2: Widok szczegółów aukcji z wizualizacją 3D

8.2.3. System nawigacji

Oba widoki zawierają identyczny, minimalistyczny pasek nawigacji z logo „AuctionHub” po lewej stronie oraz menu zawierające linki: „Strona Główna”, „Aukcje na żywo”, „Przeglądanie 3D”, „Kategorie”, „Domy Aukcyjne”. Po prawej stronie paska znajdują się przyciski „Zaloguj się” oraz wyróżniony przycisk „Zarejestruj się”.

8.2.4. Strony logowania i rejestracji

Aplikacja zawiera również dedykowane widoki do autoryzacji użytkowników. Strony logowania i rejestracji utrzymane są w spójnej stylistyce z resztą aplikacji, wykorzystując to samo tło marmurowe oraz transparentne panele formularzy. Formularze zawierają standardowe pola (email, hasło) oraz przyciski akcji w charakterystycznym niebieskim kolorze. Design tych stron jest minimalistyczny i skupiony na funkcjonalności, zapewniając użytkownikom szybki i bezproblemowy proces uwierzytelnienia.



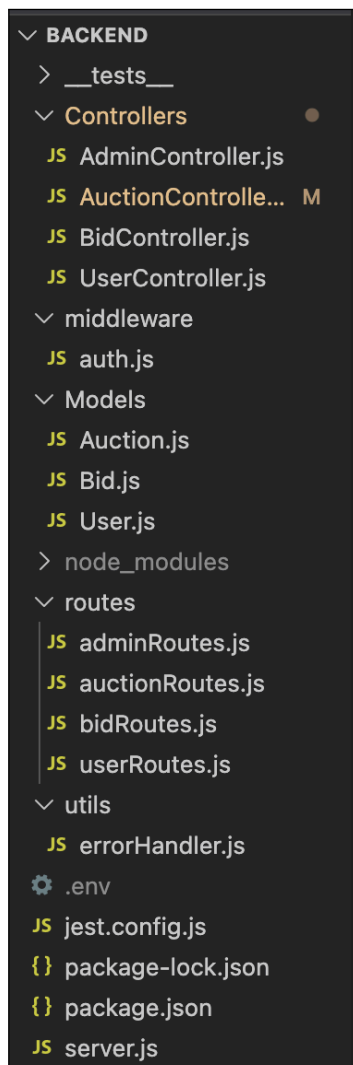
Rysunek 8.3: Widok strony logowania

9. Implementacja bazy danych oraz logiki biznesowej

9.1. Struktura części serwerowej

Na rysunku 9.1 przedstawiono strukturę części serwerowej systemu aukcyjnego:

- tests - w tym folderze zawarte są testy: jednostkowe, implementacyjne i funkcjonalne, opisane w rozdziale x.
- Controllers - w tym folderze znajduje się kod opisujący logikę biznesową.
- Middleware - w tym folderze znajduje się kod opisujący autentykację za pomocą JWT.
- Models - w tym folderze znajduje się kod opisujący modele: User, Bid i Auction dla bazy danych.
- Routes - w tym folderze są opisane trasy URL.
- Utils - w tym folderze są opisane funkcje pomocnicze.



Rysunek 9.1: Struktura części serwerowej systemu aukcyjnego

9.2. Połączenie z bazą danych

W celu uproszczenia pracy z projektem zostało użyte rozwiązanie Atlas, czyli rozwiązanie chmurowe bazy danych MongoDB. Dzięki takiemu działaniu udało się ułatwić pracę z projektem. Serwer łączy się z bazą danych, za pomocą zmiennej środowiskowej. Jeżeli połączenie nie nastąpi, wyświetlany jest typ błędu.

```
1  const connectDB = async () => {
2    try {
3
4      const conn = await mongoose.connect(process.env.MONGODB_URI, {
5        serverSelectionTimeoutMS: 5000,
6      });
7      console.log(`MongoDB Connected: ${conn.connection.host}`);
8      console.log(`Database: ${conn.connection.name}`);
9    } catch (error) {
10     console.error('MongoDB Connection Error:', error.message);
11     process.exit(1);
12   }
13 }
```

Listing 9.1: Konfiguracja połączenia z bazą danych

9.3. Rejestracja użytkowników

Aby w pełni korzystać z systemu aukcyjnego, użytkownicy muszą najpierw dokonać rejestracji. Użytkownik obowiązkowo podaje nazwę użytkownika, email, hasło, a opcjonalnie może podać imię, nazwisko oraz numer telefonu. Jeżeli użytkownik podał istniejący email bądź nazwę użytkownika, musi dokonać ich zmiany. Jeżeli wszystkie pola są wprowadzone prawidłowo, wtedy tworzymy model User i token JWT oraz zapisujemy użytkownika w bazie danych. Hasła są hashowane przed zapisem do bazy danych, w celu zabezpieczenia użytkownika. Jeżeli wystąpi jakiś błąd podczas rejestracji, cały ten proces jest anulowany. Na listingu przedstawiono kod dotyczący rejestracji użytkowników. Na listingu 9.2 przedstawiono ochronę haseł poprzez hashowanie.

```
1  export const register = async (req, res) => {
2    try {
3      const { username, email, password,
4        firstName, lastName, phone } = req.body;
5
6      if (!username || !email || !password) {
7        return res.status(400).json({
8          success: false,
```

```

9
message: 'Podaj wymagane pola: username, email i password'
10   });
11 }
12 const existingUser = await User.findOne({
13   $or: [{ email }, { username }] });
14 if (existingUser) {
15   return res.status(400).json({
16     success: false,
17     message: existingUser.email === email
18       ? 'Uzytkownik z tym adresem email juz istnieje'
19       : 'Nazwa uzytkownika jest juz zajeta'
20   });}
21 const newUser = await User.create({
22   username, email,
23   password, firstName,
24   lastName, phone
25 });
26
27 const verificationToken = newUser.createVerificationToken();
28 await newUser.save({ validateBeforeSave: false });
29 console.log('Verification token:', verificationToken);
30 createSendToken(newUser, 201,
31   res, 'Konto zostalo utworzone pomyslnie');
32
33 } catch (error) {
34   if (error.code === 11000) {
35     const field = Object.keys(error.keyPattern)[0];
36     return res.status(400).json({
37       success: false,
38       message: `${field} === 'email' ? 'Email' :
39         'Nazwa uzytkownika'} jest juz zajeta`
40     });
41   }
42   return sendErrorResponse(res, 500,
43     'Blad podczas rejestracji', error);
44 }
45 };

```

Listing 9.2: Implementacja procesu rejestracji użytkownika

9.4. Logowanie użytkownika

Po dokonaniu rejestracji, użytkownik może się zalogować do systemu aukcyjnego. Przy logowaniu wprowadza email bądź nazwę użytkownika oraz podane hasło podczas rejestracji. Podczas procesu logowania porównujemy hashe hasła, które zostało wprowadzone przez użytkownika, z haszem zapisanym w bazie danych. Dzięki takiej praktyce podczas wycieku danych z bazy, użytkownicy nie tracą swojego loginu i hasła. Na listingu 9.3 przedstawiono kod, który obsługuje logowanie użytkowników.

```
1 export const login = async (req, res) => {
2   try {
3     const { credential, password } = req.body;
4
5     if (!credential || !password) {
6       return res.status(400).json({
7         success: false,
8         message: 'Podaj email/username i hasło'
9       });
10    }
11    const user = await User.findByCredential(credential);
12    if (!user || !(await user.comparePassword(password))) {
13      return res.status(401).json({
14        success: false,
15        message: 'Nieprawidłowe dane logowania'
16      });
17    }
18    if (!user.isActive) {
19      return res.status(401).json({
20        success: false,
21        message: 'To konto zostało dezaktywowane'
22      });
23    }
24    createSendToken(user, 200, res, 'Zalogowano pomyślnie');
25  } catch (error) {
26    return sendErrorResponse(res, 500,
27      'Błąd podczas logowania', error);
28  }
29 };
```

Listing 9.3: Implementacja procesu logowania użytkownika

9.5. Autentykacja użytkownika

Użytkownicy są autentykowani poprzez Json Web Token. Ten standard pozwala na bezpieczną wymianę danych pomiędzy klientem a serwerem jako obiekt JSON oraz na kontrolowanie sesji użytkowników. JSON Web Token składa się z 3 części: nagłówka, ładunku oraz podpisu. Nagłówek zawiera typ tokenu. Na listingu 9.4 przedstawiono implementację tokenów JWT.

```
1  const signToken = (id) => {
2    return jwt.sign({ id }, process.env.JWT_SECRET, {
3      expiresIn: process.env.JWT_EXPIRES_IN || '7d'
4    });
5  };
6
7  const createSendToken =
8    (user, statusCode, res, message = 'Sukces') => {
9
10   const token = signToken(user._id);
11   const cookieOptions = {
12     expires: new Date(
13       Date.now() + (process.env.JWT_COOKIE_EXPIRES_IN || 7)
14         * 24 * 60 * 60 * 1000
15     ),
16     httpOnly: true,
17   };
18
19   res.cookie('jwt', token, cookieOptions);
20
21   user.password = undefined;
22
23   res.status(statusCode).json({
24     success: true,
25     message,
26     token,
27     data: { user }
28   });
29  };
```

Listing 9.4: Implementacja autentykacji użytkownika

9.6. Zarządzanie aukcjami

9.6.1. Stworzenie aukcji

Zalogowany użytkownik może stworzyć wiele aukcji. Swojej aukcji nie może licytować, aby nie zawyżać sztucznie ceny. Przy utworzeniu każdej aukcji przez danego użytkownika, inkrementujemy liczbę wszystkich utworzonych aukcji. Na listingu 9.5 przedstawiono funkcję tworzącą aukcję.

```
1 export const createAuction = async (req, res) => {
2   try {
3     const auctionData = {
4       ...req.body,
5       seller: req.user._id
6     };
7
8     const startTime = new Date(auctionData.startTime || Date.now());
9     const endTime = new Date(auctionData.endTime);
10
11    if (endTime <= startTime) {
12      return res.status(400).json({
13        success: false,
14        message: 'Data zakonczenia musi byc
15        pozniejsza niz data rozpoczecia'
16      });
17    }
18
19    const auction = await Auction.create(auctionData);
20    await User.findByIdAndUpdate(req.user._id, {
21      $inc: { 'stats.totalAuctionsCreated': 1 }
22    });
23
24    res.status(201).json({
25      success: true,
26      message: 'Aukcja zostala utworzona pomyslnie',
27      data: auction
28    });
29  } catch (error) {
30    if (error.name === 'ValidationError') {
31      const messages = Object.values(error.errors)
32        .map(err => err.message);
33      return res.status(400).json({
34        success: false,
```

```

34     message: messages[0]
35   });
36 }
37 res.status(500).json({
38   success: false,
39   message: 'Bład podczas tworzenia aukcji',
40   error: error.message
41 });
42 }
43 };

```

Listing 9.5: Obsługa utworzenia aukcji

9.6.2. Kategorie aukcji

Każda aukcja ma przypisaną jedną z dwunastu kategorii. Dzięki temu podczas wyświetlania wszystkich aukcji możemy filtrować dane w taki sposób, aby użytkownik miał łatwość w szukaniu potrzebnego dla siebie przedmiotu. Listing 9.6 przedstawia kod odpowiadający za opisanie kategorii aukcji.

```

1  export const getCategories = async (req, res) => {
2    try {
3      const categories = [
4        'Sztuka', 'Antyki', 'Bizuteria',
5        'Monety i Banknoty', 'Ksiazki', 'Militaria',
6        'Meble', 'Porcelana i Ceramika', 'Zegarki',
7        'Instrumenty Muzyczne', 'Elektronika', 'Inne',
8      ];
9
10     const categoriesWithCount = await Promise.all(
11       categories.map(async (category) => {
12         const count = await Auction.countDocuments({
13           category,
14           status: 'active',
15           isDeleted: false,
16           endTime: { $gt: Date.now() }
17         });
18         return { name: category, count };
19       })
20     );
21     res.status(200).json({
22       success: true,
23       data: categoriesWithCount

```

```

24     });
25   } catch (error) {
26     res.status(500).json({
27       success: false,
28       message: 'Błąd podczas pobierania kategorii',
29       error: error.message
30     });
31   }
32 };

```

Listing 9.6: Obsługa wyświetlania aukcji

9.6.3. Licytowanie aukcji

Użytkownik może licytować wiele aukcji na raz. Każda aukcja ma cenę wywoławczą i najmniejszą cenę o jaką można podbić aukcję. Po utworzeniu zgodnego podbicia, aktualizujemy cenę aukcji, poprzednie podbicia ustawiamy jako przebite i zatwierdzamy transakcję. Na listingu 9.7 został przedstawiony proces licytowania aukcji przez jednego użytkownika.

```

1  export const placeBid = async (req, res) => {
2    const mongoose = (await import('mongoose')).default;
3    const session = await mongoose.startSession();
4    session.startTransaction();
5
6    try {
7      const { amount } = req.body;
8      const auctionId = req.params.id;
9      const bidderId = req.user._id;
10
11      const auction = await Auction.findById(auctionId)
12        .session(session);
13
14      const bidArray = await Bid.create([
15        {
16          auction: auctionId,
17          bidder: bidderId,
18          amount,
19          previousBid: auction.currentPrice,
20          isWinning: true,
21          status: 'active'
22        }
23      ], { session });
24
25      const minimumBid = auction.currentPrice + auction.bidIncrement;

```

```

24         if (amount < minimumBid) {
25
26             throw new Error(`Minimalna oferta to ${minimumBid} zł`);
27         }
28
29         const bid = bidArray[0];
30         await Auction.findOneAndUpdate(
31             { _id: auctionId, __v: auction.__v },
32             {
33                 $set: {
34                     currentPrice: amount,
35                     winnerId: bidderId
36                 },
37                 $inc: {
38                     totalBids: 1,
39                     __v: 1
40                 }
41             },
42             { new: true, session, runValidators: true }
43         );
44
45         await Bid.updateMany(
46             {
47                 auction: auctionId,
48                 _id: { $ne: bid._id },
49                 isWinning: true
50             },
51             {
52                 $set: {
53                     isWinning: false,
54                     status: 'outbid'
55                 }
56             },
57             { session }
58         );
59
60         await session.commitTransaction();
61
62         const populatedBid = await Bid.findById(bid._id)
63             .populate('bidder', 'username rating')
64             .populate('auction', 'title currentPrice endTime');

```

```

65     res.status(201).json({
66         success: true,
67         message: 'Oferta została złożona pomyślnie',
68         data: populatedBid
69     });
70
71 } catch (error) {
72     await session.abortTransaction();
73     res.status(500).json({
74         success: false,
75         message: error.message || 'Błąd podczas składania oferty'
76     });
77 } finally {
78     session.endSession();
79 }
80 };

```

Listing 9.7: Proces licytowania aukcji

9.6.4. Wyświetlanie wszystkich aukcji

Niealogowani użytkownicy mogą wyświetlać wszystkie aukcje. Aukcje mogą być sortowane po kategorii, cenie albo po czasie aukcji. Na listingu 9.8 przedstawiono funkcję, która pozwala na pobranie wszystkich aukcji.

```

1  export const getAllAuctions = async (req, res) => {
2      try {
3          const {
4              page = 1,
5              limit = 20,
6              category,
7              minPrice,
8              maxPrice,
9              sortBy = 'endTime',
10             sortOrder = 'asc',
11             status = 'active'
12         } = req.query;
13
14         const query = {
15             isDeleted: false
16         };
17         if (category) query.category = category;
18

```

```

19     if (minPrice || maxPrice) {
20         query.currentPrice = {};
21
22         if (minPrice) query.currentPrice.$gte = parseFloat(minPrice);
23
24         if (maxPrice) query.currentPrice.$lte = parseFloat(maxPrice);
25     }
26
27     if (status === 'active') {
28         query.endTime = { $gt: Date.now() };
29     }
30
31     const skip = (parseInt(page) - 1) * parseInt(limit);
32
33     const auctions = await Auction.find(query)
34         .populate('seller', 'username rating')
35         .populate('winnerId', 'username')
36         .sort({ [sortBy]: sortOrder === 'desc' ? -1 : 1 })
37         .limit(parseInt(limit))
38         .skip(skip);
39
40     const total = await Auction.countDocuments(query);
41
42     res.status(200).json({
43         success: true,
44         results: auctions.length,
45         totalPages: Math.ceil(total / parseInt(limit)),
46         currentPage: parseInt(page),
47         total,
48         data: auctions
49     });
50 } catch (error) {
51     res.status(500).json({
52         success: false,
53         message: 'Błąd podczas pobierania aukcji',
54         error: error.message
55     });
56 }

```

Listing 9.8: Obsługa mechanizmu wyświetlania wszystkich aukcji

10. Integracja

Proces integracji obejmuje kilka warstw technicznych: kompilację kodu natywnego do formatu WebAssembly przy użyciu kompilatora Emscripten, zaprojektowanie architektury komponentów React umożliwiającej efektywne zarządzanie pojedynczą instancją komponentu renderującego oraz implementację mechanizmów komunikacji między kodem JavaScript a skompilowanym kodem C++.

10.1. Kompilacja do WebAssembly

Kompilacja komponentu renderującego do formatu WebAssembly wymaga odpowiedniej konfiguracji systemu budowania CMake oraz użycia kompilatora emscripten. Proces ten przekształca kod źródłowy C++ w pliki wykonywalne w środowisku przeglądarki internetowej.

10.1.1. Konfiguracja CMake dla Emscripten

Konfiguracja projektu dla Emscripten wymaga zdefiniowania specyficznych flag kompilacji, które umożliwiają integrację z WebGPU oraz eksport funkcji dostępnych z poziomu JavaScript. Najważniejsze flagi kompilacji to:

- `-sUSE_WEBGPU=1` - włączenie wsparcia dla API WebGPU w skompilowanym module,
- `-sASYNCIFY` - umożliwienie asynchronicznego wykonywania kodu, niezbędne dla operacji WebGPU,
- `-sEXPORTED_FUNCTIONS` - lista funkcji C++ eksportowanych do JavaScript,
- `-sEXPORTED_RUNTIME_METHODS` - metody runtime Emscripten dostępne z poziomu JavaScript,
- `-sFETCH` - wsparcie dla asynchronicznego pobierania zasobów,
- `-sALLOW_MEMORY_GROWTH=1` - dynamiczne rozszerzanie pamięci WebAssembly.

10.1.2. Generowane pliki wyjściowe

Proces kompilacji generuje dwa główne pliki:

- `main.wasm` - binarny moduł WebAssembly zawierający skompilowany kod C++,
- `main.js` - kod JavaScript pozwalający na wywoływanie formatu WebAssembly. Odpowiedzialny za inicjalizację modułu, zarządzanie pamięcią oraz mostkowanie wywołań między JavaScript a WebAssembly.

Plik `main.js` definiuje globalny obiekt `Module`, który stanowi interfejs komunikacji z kodem `WebAssembly`. Obiekt ten zawiera atrybuty, które kod wygenerowany przez `Emscripten` wywołuje w różnych momentach swojego działania.

10.2. Architektura integracji w React

Integracja komponentu `WebGPU` z aplikacją `React` opiera się na wzorcu `Provider`, który zapewnia współdzielenie pojedynczej instancji komponentu renderującego między wieloma komponentami interfejsu użytkownika. Architektura ta rozwiązuje problem ponownej inicjalizacji kosztownego kontekstu `WebGPU` przy nawigacji między widokami aplikacji.

10.2.1. Wzorzec Provider

Komponent `WebGPUCanvasProvider` implementuje wzorzec `Provider` z `React Context API`. Działa jako singleton zarządzający cyklem życia modułu `WebAssembly` oraz elementem canvas, na którym wykonywane jest renderowanie. `Provider` opakowuje całą aplikację, zapewniając dostęp do API komponentu renderującego z dowolnego miejsca w drzewie komponentów.

```
1 export function WebGPUCanvasProvider({ children }) {
2   const [moduleReady, setModuleReady] = useState(false)
3   const [isLoading, setIsLoading] = useState(true)
4   const [error, setError] = useState(null)
5   const [currentModel, setCurrentModel] = useState('fourareen')
6
7   // ... logika zarządzania modulem WASM
8   return (
9     <WebGPUCanvasContext.Provider value={contextValue}>
10      {children}
11      {/* Wspoldzielony canvas renderowany przez Portal */}
12    </WebGPUCanvasContext.Provider>
13  )
14 }
```

Listing 10.1: Struktura komponentu `Provider`

10.2.2. Hooki dostępu do API

Dostęp do funkcjonalności komponentu renderującego z komponentów potomnych realizowany jest przez dedykowane hooki `React`:

- `useWebGPUCanvas()` - publiczny hook zwracający API do kontroli komponentu renderującego (zmiana modelu, status ładowania, informacje o błędach),

- `useWebGPUCanvasInternal()` - wewnętrzny hook używany przez komponent `WebGPUCanvas` do zarządzania slotami wyświetlania.

```

1  const publicApi = {
2    moduleReady,    // czy moduł WASM jest zainicjalizowany
3    isLoading,      // czy trwa ładowanie
4    error,          // komunikat błędu (jeśli wystąpił)
5    currentModel,   // nazwa aktualnie wyświetlanego modelu
6    changeModel,    // funkcja zmiany modelu 3D
7    modelNames      // lista dostępnych modeli
8  }

```

Listing 10.2: Publiczne API komponentu renderującego

10.3. Mechanizm React Portals

Kluczowym elementem architektury jest wykorzystanie mechanizmu React Portals do dynamicznego przenoszenia elementu canvas między różnymi miejscami w strukturze DOM. Pozwala to na wyświetlanie tego samego komponentu renderującego 3D w różnych sekcjach aplikacji bez konieczności reinicjalizacji kontekstu WebGPU.

10.3.1. Zasada działania

React Portal umożliwia renderowanie komponentu potomnego do węzła DOM znajdującego się poza hierarchią rodzica. W przypadku integracji WebGPU, współdzielony canvas jest renderowany przez Portal do aktualnie aktywnego „slotu” - kontenera DOM wskazanego przez komponent `WebGPUCanvas`.

```

1  {hostNode
2    ? createPortal(
3      <SharedWebGPUCanvas
4        options={presentationOptions}
5        isLoading={isLoading}
6        error={error}
7        moduleReady={moduleReady}
8      />,
9      hostNode
10    )
11  : null}

```

Listing 10.3: Renderowanie canvas przez Portal

10.3.2. Zarządzanie slotami

System slotów pozwala komponentom `WebGPUCanvas` rezerwować miejsce wyświetlania komponentu renderującego. Gdy komponent jest montowany, rejestruje swój kontener jako aktywny slot. Gdy jest odmontowywany, slot jest zwalniany, a canvas może zostać przeniesiony do innego miejsca lub ukryty w fallbackowym kontenerze.

Funkcje zarządzania slotami:

- `attachSlot(slotId, container, options)` - rejestracja nowego slotu z opcjami wyświetlania (wymiary, model),
- `detachSlot(slotId)` - zwolnienie slotu przy odmontowaniu komponentu.

10.4. Ładowanie modułu WebAssembly

Inicjalizacja modułu `WebAssembly` odbywa się asynchronicznie podczas pierwszego montowania komponentu canvas. Proces ten obejmuje wstrzyknięcie skryptu `main.js`, konfigurację obiektu `Module` oraz obsługę stanów ładowania.

10.4.1. Konfiguracja obiektu Module

Przed załadowaniem skryptu `Emscripten`, konfigurowany jest globalny obiekt `window.Module` z callbackami obsługującymi zdarzenia cyklu życia:

```
1  const moduleConfig = {
2    canvas: canvasRef.current,
3
4    onRuntimeInitialized: () => {
5      setModuleReady(true)
6      setIsLoading(false)
7    },
8
9    onAbort: (what) => {
10     setError('WebAssembly initialization failed')
11     setIsLoading(false)
12   },
13
14   print: (text) => console.log('WASM:', text),
15   printErr: (text) => console.error('WASM Error:', text)
16 }
17
18 window.Module = moduleConfig
```

Listing 10.4: Konfiguracja obiektu `Module`

10.4.2. Asynchroniczne wstrzykiwanie skryptu

Skrypt `main.js` jest dynamicznie dodawany do dokumentu HTML. Mechanizm zapewnia, że skrypt jest ładowany tylko raz, niezależnie od liczby montowań komponentu:

```
1  if (!globalState.scriptAppended) {
2    const script = document.createElement('script')
3    script.src = '/main.js'
4    script.async = true
5    script.onerror = () => {
6      setError('Failed to load WebAssembly runtime')
7      setIsLoading(false)
8    }
9
10   document.body.appendChild(script)
11   globalState.scriptAppended = true
12 }
```

Listing 10.5: Dynamiczne ładowanie skryptu Emscripten

10.5. Wirtualny system plików

Emscripten udostępnia wirtualny system plików (VFS), który emuluje operacje plikowe w środowisku przeglądarki. Integracja wykorzystuje IDBFS - backend oparty na IndexedDB - do persystentnego przechowywania zasobów między sesjami użytkownika.

10.5.1. Montowanie systemu plików

Podczas fazy `preRun` konfigurowane są punkty montowania dla persystentnych danych:

```
1  moduleConfig.preRun.push(() => {
2    const FS = window.FS
3    const IDBFS = window.IDBFS
4
5    // Katalog na baze danych modeli
6    FS.mkdir('/database')
7    FS.mount(IDBFS, {}, '/database')
8
9    // Katalog na cache zasobow
10   FS.mkdir('/resources_cache')
11   FS.mount(IDBFS, {}, '/resources_cache')
12 })
```

Listing 10.6: Konfiguracja IDBFS

Synchronizacja z IndexedDB wykonywana jest przez funkcję `FS.syncfs()`, która może działać w trybie odczytu (z IndexedDB do VFS) lub zapisu (z VFS do IndexedDB).

10.6. Komunikacja JavaScript-C++

Komunikacja między warstwą JavaScript a skompilowanym kodem C++ odbywa się przez funkcje eksportowane z modułu `WebAssembly`. Funkcje te są dostępne jako metody obiektu `window.Module`.

10.6.1. Eksportowane funkcje

Komponent renderujący eksportuje następujące funkcje:

- `change_model(modelName)` - zmiana wyświetlanego modelu 3D,
- `resize_canvas(width, height)` - aktualizacja wymiarów renderowania,
- `set_resource_base(path)` - konfiguracja ścieżki bazowej dla zasobów.

```
1  const changeModel = useCallback((modelName) => {
2    if (!moduleReady) {
3      console.warn('WebGPU module not ready yet.')
4      return false
5    }
6
7    if (typeof window.Module.change_model !== 'function') {
8      setError('WebAssembly module missing required function')
9      return false
10   }
11
12   try {
13     window.Module.change_model(modelName)
14     setCurrentModel(modelName)
15     return true
16   } catch (err) {
17     setError('Failed to change model')
18     return false
19   }
20 }, [moduleReady])
```

Listing 10.7: Wywołanie funkcji eksportowanej z C++

10.6.2. Przekazywanie elementu canvas

Element HTML canvas jest przekazywany do modułu WebAssembly przez właściwość `Module.canvas`. Emscripten automatycznie używa tego elementu do utworzenia kontekstu WebGPU:

```
1 moduleConfig.canvas = canvasRef.current
2
3 // Po inicjalizacji modul automatycznie używa przypisanego canvas
4 // do renderowania przez WebGPU
```

Listing 10.8: Przypisanie canvas do modułu

10.7. Komponent WebGPUCanvas

Komponent `WebGPUCanvas` stanowi publiczny interfejs do osadzania komponentu renderującego 3D w dowolnym miejscu aplikacji. Obsługuje responsywne wymiarowanie oraz automatyczną zmianę modelu.

```
1 <WebGPUCanvas
2   width={800}
3   height={600}
4   model="fourareen"
5   showControls={true}
6   onModelChange={
7     (model) => console.log('Model changed:', model)
8   }
9 />
```

Listing 10.9: Użycie komponentu `WebGPUCanvas`

Komponent obsługuje trzy tryby wymiarowania:

- określone wymiary (`width/height`) - stały rozmiar w pikselach,
- proporcjonalne - podanie tylko wysokości automatycznie oblicza szerokość (proporcja 4:3),
- responsywne - brak wymiarów powoduje dopasowanie do kontenera rodzica z użyciem `ResizeObserver`.

11. Pomiar wydajności

Istotnym elementem oceny jakości oprogramowania jest analiza wydajności kluczowych operacji. W przypadku komponentu renderującego, jedną z najbardziej czasochłonnych operacji jest ładowanie trójwymiarowych modeli z plików w formacie OBJ. W niniejszym rozdziale przedstawiono metodykę pomiaru oraz wyniki analizy wydajności funkcji odpowiedzialnej za wczytywanie geometrii.

11.1. Metodyka pomiaru

Do przeprowadzenia pomiarów wykorzystano bibliotekę Google Benchmark, która umożliwia precyzyjne i powtarzalne mierzenie czasu wykonania funkcji w języku C++ [16]. Wyniki zostały zwizualizowane przy pomocy biblioteki Matplotlib [22]. Każdy test został wykonany dziesięciokrotnie, co pozwoliło na obliczenie miar statystycznych: średniej arytmetycznej, mediany oraz odchylenia standardowego.

Badaną funkcją była `load : geometry_obj()`, której zadaniem jest wczytanie pliku OBJ zawierającego trójwymiarowy model i przekształcenie go do formatu atrybutów wierzchołków. Funkcja ta wykonuje dwie główne operacje:

1. **Parsowanie pliku** – odczytanie i interpretacja danych z pliku tekstowego OBJ przy użyciu biblioteki `tinyobjloader`.
2. **Obliczenie macierzy TBN** – wyliczenie wektorów stycznych (Tangent), bistycznych (Bitangent) i normalnych (Normal) dla każdego trójkąta, niezbędnych do prawidłowego mapowania normalnych w procesie renderowania.

11.1.1. Macierz TBN

Macierz TBN (Tangent-Bitangent-Normal) jest macierzą transformacji 3×3 , która przekształca wektory z przestrzeni stycznej tekstury do przestrzeni świata [17]. Jest ona niezbędna do prawidłowego stosowania map normalnych (ang. *normal mapping*) w procesie renderowania. Dla trójkąta o wierzchołkach P_0, P_1, P_2 z odpowiadającymi współrzędnymi tekstury $(u_0, v_0), (u_1, v_1), (u_2, v_2)$, wektory krawędzi definiuje się jako:

$$\mathbf{E}_1 = \mathbf{P}_1 - \mathbf{P}_0, \quad \mathbf{E}_2 = \mathbf{P}_2 - \mathbf{P}_0 \quad (1)$$

$$\Delta u_1 = u_1 - u_0, \quad \Delta v_1 = v_1 - v_0 \quad (2)$$

$$\Delta u_2 = u_2 - u_0, \quad \Delta v_2 = v_2 - v_0 \quad (3)$$

Wektor styczny $\mathbf{T}$ i bistyczny $\mathbf{B}$ obliczane są ze wzorów:

$$\mathbf{T} = \text{normalize}(\mathbf{E}_1 \cdot \Delta v_2 - \mathbf{E}_2 \cdot \Delta v_1) \quad (4)$$

$$\mathbf{B} = \text{normalize}(\mathbf{E}_2 \cdot \Delta u_1 - \mathbf{E}_1 \cdot \Delta u_2) \quad (5)$$

Następnie wykonywana jest ortogonalizacja Grama-Schmidta, aby zapewnić ortogonalność wektorów względem wektora normalnego $\mathbf{N}$:

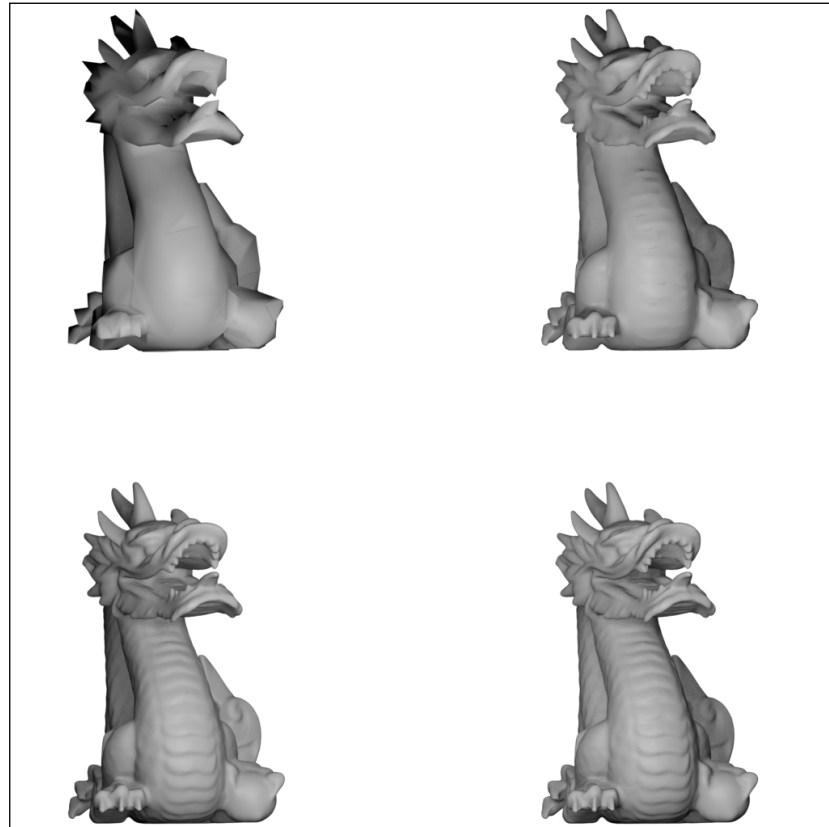
$$\mathbf{T}' = \text{normalize}(\mathbf{T} - (\mathbf{T} \cdot \mathbf{N}) \cdot \mathbf{N}) \quad (6)$$

$$\mathbf{B}' = \mathbf{N} \times \mathbf{T}' \quad (7)$$

11.1.2. Dane testowe

W badaniu wykorzystano model 3D *Stanford Dragon* – klasyczny model testowy powszechnie stosowany w badaniach grafiki komputerowej [4]. Model ten został przygotowany w czterech wariantach o różnej złożoności geometrycznej:

- 1 tysiąc wierzchołków
- 10 tysięcy wierzchołków
- 100 tysięcy wierzchołków
- ok. 1 milion wierzchołków



Rysunek 11.1: Model 3D Stanford Dragon w czterech wariantach złożoności geometrycznej: lewa góra – 1k, prawa góra – 10k, lewy dół – 100k, prawy dół – 1M wierzchołków

11.1.3. Środowisko testowe

Pomiary zostały wykonane na komputerze wyposażonym w czterordzeniowy procesor o taktowaniu 2712 MHz z następującą hierarchią pamięci podręcznej:

- L1 Dane: 32 KiB (x4)
- L1 Instrukcje kodu: 32 KiB (x4)
- L2 Połączony: 256 KiB (x4)
- L3 Połączony: 6144 KiB (x1)

11.2. Wyniki pomiarów

W tabeli 11.1 przedstawiono zestawienie wyników pomiarów dla wszystkich wariantów modelu. Zmierzono trzy różne metryki czasowe: pełny czas ładowania (czas rzeczywisty, ang. *wall clock time*), czas procesora CPU oraz czas samego parsowania pliku.

Tabela 11.1: Wyniki pomiarów wydajności ładowania modeli OBJ

Metryka	1k	10k	100k	1M
Pełne ładowanie (ms)	5,05	43,1	427	4290
Czas CPU (ms)	4,87	42,0	421	4282
Tylko parsowanie (ms)	4,16	38,0	392	3927
Odch. std. pełnego ład. (ms)	0,178	0,502	2,34	32,8
Przepustowość (mln wierzch./s)	1,19	1,39	1,40	1,38

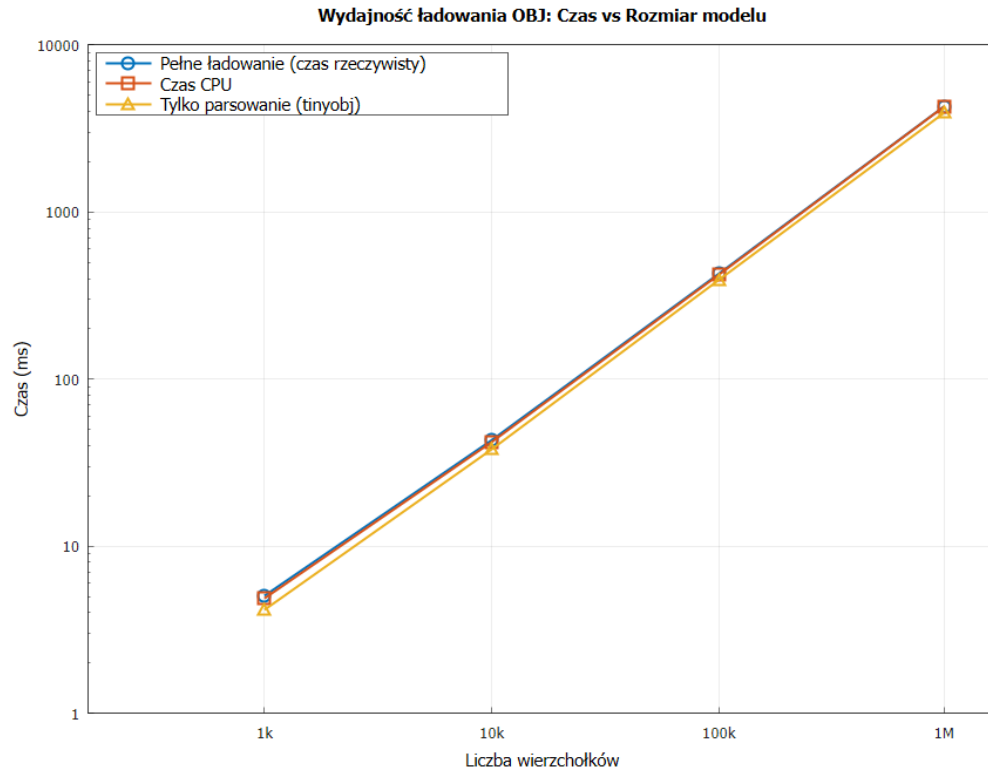
11.2.1. Analiza skalowania czasowego

Na rysunku 11.2 przedstawiono zależność czasu wykonania od rozmiaru modelu 3D w skali podwójnie logarytmicznej. Liniowy charakter wykresu w tej skali potwierdza, że złożoność czasowa algorytmu jest proporcjonalna do liczby wierzchołków, tj. wynosi $O(n)$, gdzie n oznacza liczbę wierzchołków.

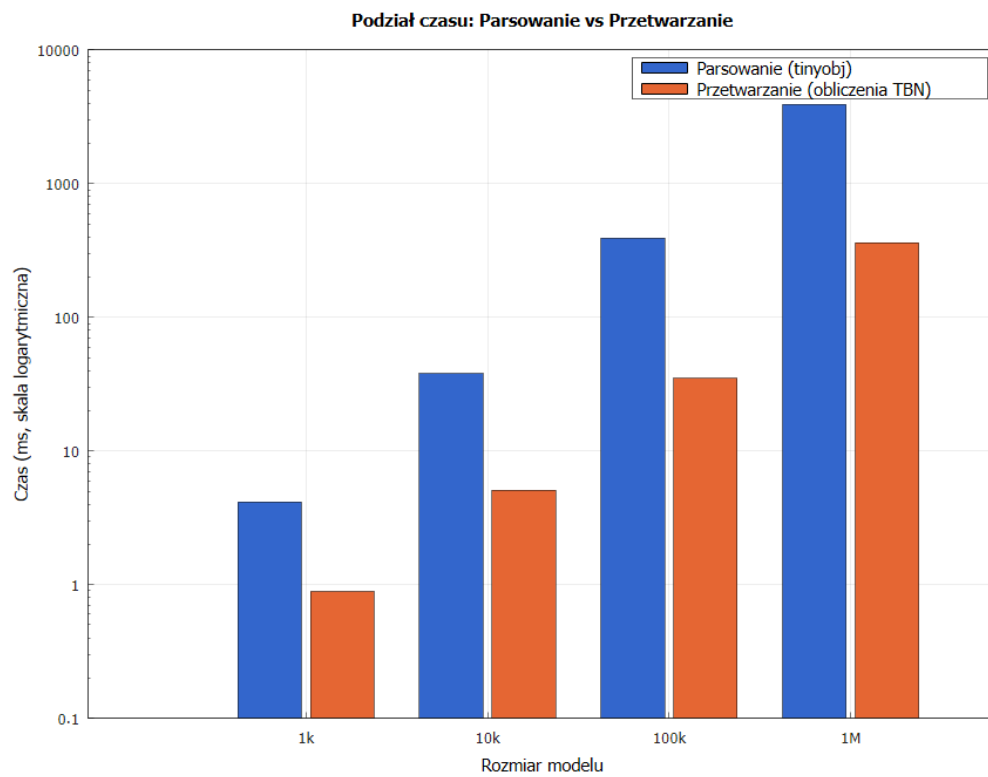
Dla dziesięciokrotnego zwiększenia liczby wierzchołków czas wykonania rośnie około dziesięciokrotnie, co jest zgodne z oczekiwaniami dla algorytmu o liniowej złożoności obliczeniowej.

11.2.2. Podział czasu wykonania

Szczególnie istotna jest analiza podziału czasu między dwie główne fazy operacji: parsowanie pliku oraz obliczenia macierzy TBN. Na rysunku 11.3 przedstawiono ten podział dla poszczególnych rozmiarów modeli.



Rysunek 11.2: Zależność czasu ładowania od rozmiaru modelu (skala logarytmiczna)

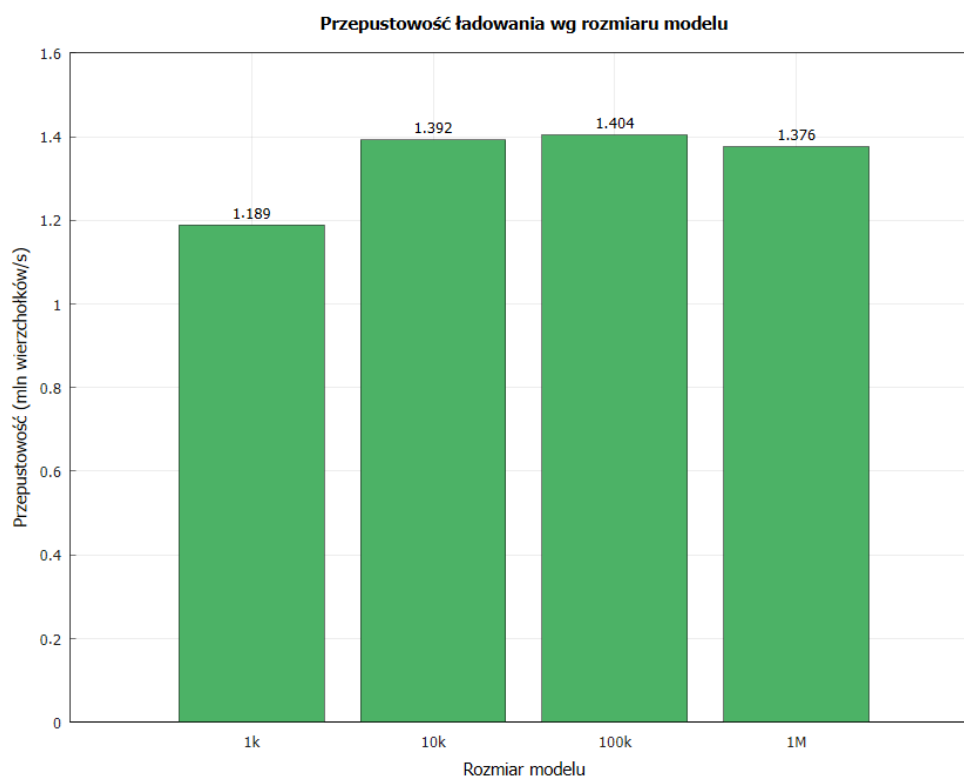


Rysunek 11.3: Podział czasu: parsowanie pliku (niebieski) vs obliczenia TBN (pomarańczowy)

Obliczenia macierzy TBN stanowią stały procent całkowitego czasu ładowania – około 8-9% dla wszystkich rozmiarów modeli. Oznacza to, że dominującym czynnikiem wpływającym na wydajność jest operacja parsowania pliku OBJ, która obejmuje odczyt z dysku oraz interpretację formatu tekstowego.

11.2.3. Przepustowość operacji ładowania

Przepustowość operacji ładowania, wyrażona jako liczba przetworzonych wierzchołków na sekundę, utrzymuje się na stabilnym poziomie około 1,4 miliona wierzchołków na sekundę dla modeli powyżej 10 tysięcy wierzchołków (rysunek 11.4). Niższa przepustowość dla najmniejszego modelu 3D s(1,19 mln wierzch./s) wynika z narzutu związanego z inicjalizacją operacji wejścia/wyjścia.



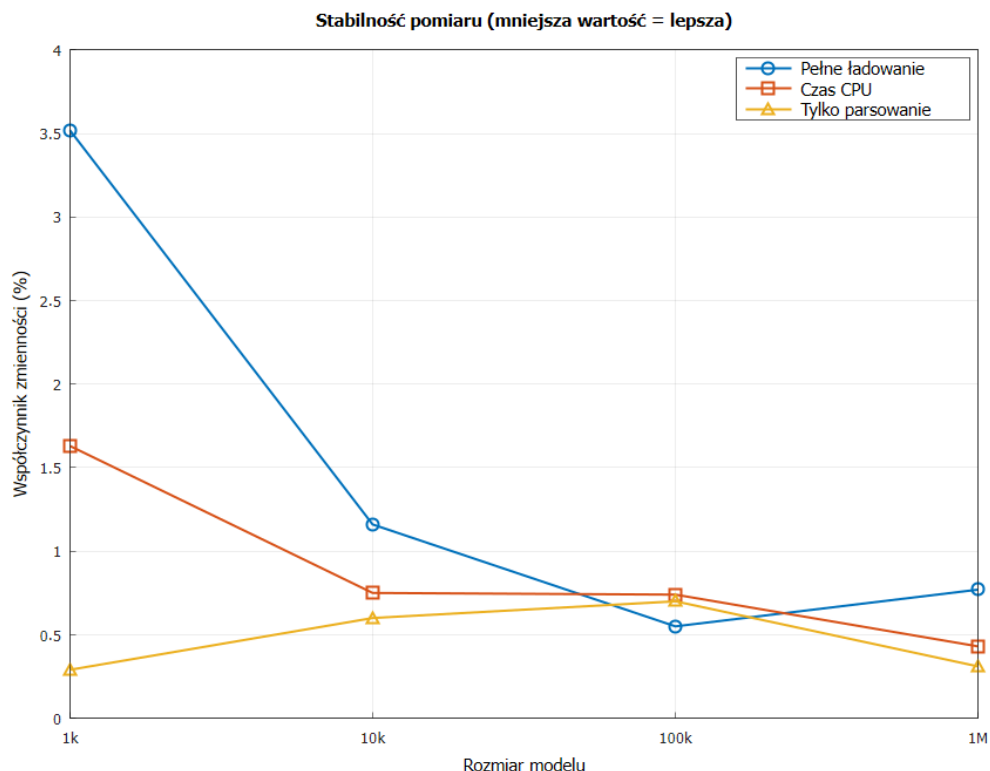
Rysunek 11.4: Przepustowość ładowania modeli OBJ

11.2.4. Analiza stabilności pomiarów

W celu oceny wiarygodności wyników obliczono współczynnik zmienności (CV, ang. *Coefficient of Variation*), który jest miarą względnego rozproszenia wyników. Definiowany jest wzorem:

$$CV = \frac{\sigma}{\mu} \cdot 100\% \quad (8)$$

gdzie σ oznacza odchylenie standardowe, a μ średnią arytmetyczną. Niższa wartość współczynnika zmienności świadczy o większej stabilności i powtarzalności pomiarów [1].



Rysunek 11.5: Współczynnik zmienności pomiarów

Jak widać na rysunku 11.5, współczynnik zmienności dla wszystkich pomiarów nie przekracza 4%, przy czym dla większych modeli osiąga wartości poniżej 1%. Świadczy to o wysokiej powtarzalności i wiarygodności przeprowadzonych pomiarów. Większa zmienność dla najmniejszego modelu (1k) wynika z proporcjonalnie większego wpływu szumów pomiarowych przy krótkim czasie wykonania.

11.3. Wnioski z analizy wydajności

Przeprowadzona analiza wydajności funkcji ładowania modeli OBJ pozwala sformułować następujące wnioski:

1. **Liniowa złożoność** – algorytm charakteryzuje się złożonością czasową $O(n)$, co oznacza, że czas ładowania rośnie proporcjonalnie do liczby wierzchołków w modelu.
2. **Dominacja operacji I/O** – parsowanie pliku tekstowego stanowi około 91-92% całkowitego czasu ładowania. Optymalizacja tej fazy, np. poprzez użycie binarnego formatu plików, mogłaby znacząco poprawić wydajność.
3. **Stabilna przepustowość** – dla modeli powyżej 10 tysięcy wierzchołków przepustowość utrzymuje się na poziomie około 1,4 miliona wierzchołków na sekundę.
4. **Wysoka powtarzalność** – niski współczynnik zmienności (poniżej 1% dla dużych modeli) świadczy o deterministycznym charakterze operacji i wiarygodności pomiarów.

5. **Czas ładowania modeli produkcyjnych** – dla typowego modelu o złożoności 100 tysięcy wierzchołków czas ładowania wynosi około 427 ms, co jest akceptowalne dla jednorazowej operacji inicjalizacji sceny.

Uzyskane wyniki potwierdzają, że implementacja funkcji ładowania geometrii jest wydajna i skalowalna, spełniając wymagania aplikacji renderującej trójwymiarowe obiekty w czasie rzeczywistym.

12. Testowanie systemu aukcyjnego

W celu zapewnienia poprawnego działania serwisu aukcyjnego wykorzystano szkielet testowy oraz utworzono pakiet testów. Zawiera on testy integracyjne, jednostkowe oraz funkcjonalne. Testy integracyjne sprawdzają jak poszczególne części systemu współpracują ze sobą, takie jak rejestracja czy logowanie. Testy jednostkowe sprawdzają pojedyncze metody oraz funkcje, izolując je od reszty kodu. Testy funkcjonalne sprawdzają, czy napisany kod spełnia swoje działanie.

W celu sprawdzenia poprawności testów wykorzystano narzędzie do uruchamiania testów o nazwie Jest [19] z menadżera pakietów npm. Szkielet ten pozwala na łatwe skonfigurowanie testów oraz na ich bezproblemowe uruchomienie.

Na listingu 12.1 przedstawiono konfigurację środowiska testowego przed uruchomieniem testów. Przed uruchomieniem testów, inicjalizujemy instancję MongoDB w pamięci ram podczas testowania. Dzięki temu możemy przetestować działania na bazie. Po każdym teście wszystkie dokumenty z bazy danych są usuwane, a po uruchomieniu wszystkich testów, połączenie z bazą danych jest zrywane i serwer jest wyłączany. Dzięki takiej praktyce mamy pewność, że testy są od siebie niezależne.

```
1  beforeAll(async () => {
2      mongoServer = await MongoMemoryServer.create();
3      const mongoUri = mongoServer.getUri();
4      await mongoose.connect(mongoUri);
5  });
6
7  afterAll(async () => {
8      await mongoose.disconnect();
9      await mongoServer.stop();
10  });
11
12  afterEach(async () => {
13      await User.deleteMany({});
14  });
```

Listing 12.1: Konfiguracja środowiska testowego

12.1. Testy jednostkowe

Na listingu 12.2 przedstawiono test jednostkowy który sprawdza poprawność działania dodania użytkownika z wymaganymi przez bazę danych polami. Jeżeli któreś z tych pól jest puste, w takim przypadku test zakończył się niepowodzeniem.

```
1  test('should create user with required fields', async () => {
2      const userData = {
3          username: 'testuser',
4          email: 'test@example.com',
5          password: 'password123'
6      };
7
8      const user = await User.create(userData);
9
10     expect(user.username).toBe('testuser');
11     expect(user.email).toBe('test@example.com');
12     // Hasło powinno być hashowane
13     expect(user.password).not.toBe('password123');
14 });
```

Listing 12.2: Test jednostkowy utworzenia instancji modelu User

12.2. Testy integracyjne

Na listingu 12.3 przedstawiono test integracyjny, który sprawdza proces rejestracji, kiedy użytkownik poda adres email w złym formacie. W tym przypadku podczas rejestracji serwer wysłał żądanie o kodzie 400, które oznacza, że żądanie jest źle sformułowane.

```
1  test('should fail with invalid email format', async () => {
2      const response = await request(app)
3          .post('/api/auth/register')
4          .send({
5              username: 'testuser',
6              email: 'invalid-email',
7              password: 'password123'
8          })
9          .expect(400);
10     expect(response.body.success).toBe(false);
11 });
12 });
```

Listing 12.3: Test integracyjny rejestracji ze złym adresem email

12.3. Testy funkcjonalne

Na listingu 12.4 przedstawiono test funkcjonalny, który sprawdza filtrowanie aukcji po kategoriach. Utworzono obiekt użytkownika oraz aukcji z kategorią Zegarki. Potem sprawdzono, czy możliwe jest wyszukanie aktywnej aukcji poprzez kategorię i sprawdzono czy to jest jedyny utworzony obiekt z tej kategorii.

```
1  test('should filter auctions by category', async () => {
2    const seller = await User.create({
3      username: 'seller',
4      email: 'seller@example.com',
5      password: 'password123'
6    });
7
8    await Auction.create({
9      title: 'Vintage Watch',
10     description: 'Beautiful watch',
11     category: 'Zegarki',
12     startingPrice: 1000,
13     currentPrice: 1000,
14     seller: seller._id,
15     startTime: new Date(),
16     endTime: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
17     status: 'active'
18   });
19
20   const watchAuctions = await Auction.find({
21     category: 'Zegarki',
22     status: 'active' });
23
24   expect(watchAuctions.length).toBe(1);
25 });
```

Listing 12.4: Test funkcjonalny filtrowania przez kategorie

Na rysunku 12.1 przedstawiono skuteczność wszystkich testów zawartych w projekcie. Wszystkie napisane testy są kompilowane bezbłędnie, co oznacza, że najważniejsze części systemu aukcyjnego działają poprawnie.

```
PASS __tests__/functional/auction-flow.test.js
Auction Flow - Functional Tests
  ✓ should create auction from user to listing (400 ms)
  ✓ should handle complete bidding flow (970 ms)
  ✓ should handle auction completion with winner (635 ms)
  ✓ should allow users to watch and unwatch auctions (553 ms)
  ✓ should filter auctions by category (299 ms)
  ✓ should track user bidding statistics (596 ms)

PASS __tests__/integration/auth.controller.test.js
Authentication Controller - Implementation Tests
  User Registration
    ✓ should register a new user successfully (394 ms)
    ✓ should fail when email already exists (333 ms)
    ✓ should fail with invalid email format (15 ms)
  User Login
    ✓ should login with email successfully (590 ms)
    ✓ should fail with incorrect password (565 ms)
    ✓ should fail with non-existent user (277 ms)

PASS __tests__/unit/user.model.test.js
User Model - Unit Tests
  User Creation and Validation
    ✓ should create a valid user with required fields (343 ms)
    ✓ should fail when required fields are missing (4 ms)
    ✓ should validate email format (4 ms)
  Password Hashing
    ✓ should hash password before saving (318 ms)
    ✓ comparePassword should validate correct password (617 ms)
  User Methods
    ✓ getPublicProfile should return sanitized user data (314 ms)
    ✓ findByCredential should find user by email or username (331 ms)

PASS __tests__/unit/auction.model.test.js
Auction Model - Unit Tests
  Auction Creation and Validation
    ✓ should create a valid auction with required fields (11 ms)
    ✓ should validate category enum (11 ms)
  Auction Methods
    ✓ incrementViews should increase view count (33 ms)
    ✓ complete should update auction status to completed (9 ms)
    ✓ getFeatured should return only featured auctions (26 ms)

Test Suites: 4 passed, 4 total
Tests:       24 passed, 24 total
Snapshots:   0 total
Time:        10.948 s, estimated 12 s
Ran all test suites.
```

Rysunek 12.1: Raport z uruchomienia testów systemu aukcyjnego

13. Wnioski

Celem pracy była integracja, zaprojektowanie i implementacja komponentu renderującego obiekty 3d z częścią webową, w tym przypadku, systemem aukcyjnym.

Pierwszą częścią pracy nad systemem, było stworzenie projektu. Projekt zawierał wymagania funkcjonalne, niefunkcjonalne oraz diagramy ERD i BPMN. Wymienienie wymagań funkcjonalnych i niefunkcjonalnych pokazało zakres działania systemu. Diagram ERD ułatwił proces implementacji bazy danych, a diagramy BPMN pozwoliły na szybkie zrozumienie i wdrożenie procesów biznesowych w systemie aukcyjnym.

Kolejnym etapem pracy była implementacja komponentu renderującego. Proces ten wymagał dogłębnego zrozumienia niskopoziomowej komunikacji z procesorem graficznym poprzez API WebGPU. Zastosowanie wzorca potoku opartego na `std::expected` ze standardu C++23 pozwoliło na elegancką obsługę błędów podczas inicjalizacji trzynastu wymaganych zasobów GPU. Implementacja shaderów w języku WGSL umożliwiła realizację zaawansowanego modelu oświetlenia z mapami normalnych oraz wieloma źródłami światła. Szczególnym wyzwaniem okazała się kompilacja kodu do formatu WebAssembly przy użyciu narzędzia Emscripten, co wymagało odpowiedniej konfiguracji flag kompilacji oraz zrozumienia ograniczeń wirtualnego systemu plików przeglądarki. Dzięki temu udało się osiągnąć wydajność renderowania porównywalną z aplikacjami natywnymi, zachowując jednocześnie pełną kompatybilność z ekosystemem internetowym.

Następnie pracowaliśmy nad implementacją systemu aukcyjnego. System ten został podzielony na część internetową oraz serwerową. Część internetowa została stworzona zgodnie z początkowym projektem wizualnym. Podczas tworzenia części serwerowej bardzo pomogły diagramy BPMN i ERD, dzięki którym mogliśmy wdrożyć zaprojektowane rozwiązania.

Kończącym etapem było sprawdzenie działania aplikacji. Każda z funkcjonalności została przetestowana manualnie. Wykryte w ten sposób niepoprawne działania zostały naprawiane na bieżąco.

Cel pracy został w pełni zrealizowany. Udało nam się utworzyć w pełni działający kod, który pozwala na renderowanie obiektów 3D i opakowaliśmy go w system aukcyjny. Dzięki temu udało nam się znaleźć nowe zastosowanie renderingu w WebGPU.

Projektowanie tego systemu umożliwiło członkom grupy zdobycie praktycznego doświadczenia w pracy nad rozbudowaną aplikacją. Proces ten ukazał, jak istotna jest współpraca z wykorzystaniem zdalnego repozytorium oraz jak ważne są praktyki pisania czystego, czytelnego kodu.

Bibliografia

- [1] Brown, C. E. (1998). Coefficient of Variation. In *Applied Multivariate Statistics in Geohydrology and Related Sciences* (pp. 155–157). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-80328-4_13
- [2] CMake - oficjalna strona. <https://cmake.org>. [7.1.2026].
- [3] Co to jest token JWT? <https://nofluffjobs.com/pl/log/praca-w-it/poczatki-w-it/jwt-token-co-to-jest-jak-go-uzywac-w-autoryzacji/>. [7.1.2026].
- [4] Curless, B., & Levoy, M. (2023). A Volumetric Method for Building Complex Models from Range Images. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2* (1st ed.). Association for Computing Machinery. <https://doi.org/10.1145/3596711.3596726>
- [5] Dang, M. (2024). *3D graphics renderer for web applications using WebAssembly and WGPU* (Bachelor's thesis, Tampere University of Applied Sciences).
- [6] De Macedo, J., Abreu, R., Pereira, R., & Saraiva, J. (2022). WebAssembly versus JavaScript: Energy and Runtime Performance. *2022 International Conference on ICT for Sustainability (ICT4S)*, 24–34. <https://doi.org/10.1109/ICT4S55073.2022.00014>
- [7] Dear ImGui - repozytorium. <https://github.com/ocornut/imgui>. [7.1.2026].
- [8] Dostępność WebAssembly w przeglądarkach. <https://caniuse.com/wasm>. [7.1.2026].
- [9] Dostępność WebGPU w przeglądarkach. <https://caniuse.com/webgpu>. [7.1.2026].
- [10] E-commerce w Polsce 2024. https://gemius.com/documents/66/RAPORT_E-COMMERCE_2024.pdf. [15.1.2025].
- [11] Emscripten - oficjalna strona. <https://emscripten.org>. [7.1.2026].
- [12] Express.js - dokumentacja. <https://expressjs.com/en/starter/installing.html>. [7.1.2026].
- [13] Gerrard, G. (2019). *WebAssembly in Action: With examples using C++ and Emscripten*. Simon; Schuster.
- [14] GLFW - oficjalna strona. <https://www.glfw.org>. [7.1.2026].
- [15] GLM (OpenGL Mathematics) - repozytorium. <https://github.com/g-truc/glm>. [7.1.2026].
- [16] Google Benchmark - repozytorium. <https://github.com/google/benchmark>. [7.1.2026].
- [17] Halladay, K. (2019). Normal Mapping. In *Practical Shader Development: Vertex and Fragment Shaders for Game Developers* (pp. 181–200). Apress. https://doi.org/10.1007/978-1-4842-4457-9_10
- [18] International Organization for Standardization. (2024). *Programming languages — C++*.
- [19] Jest - dokumentacja. [16.1.2025].

- [20] Ji, X., Wang, H., Sheng, Q., Chen, D., & Min, S. (2025). Matrix transformation and projection technology in 3D graphic rendering. *International Conference on Computational Intelligence and Image Analysis (ICIIA 2025)*, 13700, 20–25.
- [21] Mardan, A. (2015). *Node.js w praktyce. Tworzenie skalowalnych aplikacji sieciowych*. Helion.
- [22] *Matplot++ - dokumentacja*. <https://alandefreitas.github.io/matplotplusplus/>. [7.1.2026].
- [23] *MongoDB - dokumentacja*. <https://www.mongodb.com/docs/>. [7.1.2026].
- [24] *Mongoose - dokumentacja*. <https://mongoosejs.com/docs/guide.html>. [7.1.2026].
- [25] *Ninja - oficjalna strona*. <https://ninja-build.org>. [7.1.2026].
- [26] *npm - dokumentacja*. <https://docs.npmjs.com>. [7.1.2026].
- [27] *Przykłady użycia emscripten*. <https://github.com/emscripten-core/emscripten/wiki/Porting-Examples-and-Demos>. [7.1.2026].
- [28] *React Router - dokumentacja*. <https://reactrouter.com/home>. [7.1.2026].
- [29] Rodríguez, A., García, P. L., Marquerie, J. S., Marques, R., & Blat, J. (2025). A Cross-Platform, WebGPU-Based 3D Engine for Real-Time Rendering and XR Applications. *Proceedings of the 30th International Conference on 3D Web Technology*, 1–9. <https://doi.org/10.1145/3746237.3746305>
- [30] *stb_image - implementacja*. https://github.com/nothings/stb/blob/master/stb_image.h. [7.1.2026].
- [31] Stefanov, S. (2017). *React w działaniu. Tworzenie stron internetowych*. Helion.
- [32] *TailwindCSS - dokumentacja*. <https://tailwindcss.com/docs/installation/using-vite>. [7.1.2026].
- [33] *tinyobjloader - repozytorium*. <https://github.com/tinyobjloader/tinyobjloader>. [7.1.2026].
- [34] *TW Elements - dokumentacja*. <https://tw-elements.com/docs/react/>. [7.1.2026].
- [35] Usta, Z. (2024). WEBGPU: A NEW GRAPHIC API FOR 3D WEBGIS APPLICATIONS. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLVIII-4/W9-2024, 377–382. <https://doi.org/10.5194/isprs-archives-XLVIII-4-W9-2024-377-2024>
- [36] *Vite - dokumentacja*. <https://vite.dev/guide/>. [7.1.2026].
- [37] *WebAssembly - oficjalna strona*. <https://webassembly.org>. [7.1.2026].
- [38] *WebGPU - implementacja Google*. <https://dawn.googlesource.com/dawn>. [7.1.2026].
- [39] *WebGPU - otwarta implementacja w języku Rust*. <https://github.com/gfx-rs/wgpu>. [7.1.2026].
- [40] *WebGPU - specyfikacja robocza W3C*. <https://www.w3.org/TR/webgpu/>. [7.1.2026].
- [41] *WGSL (WebGPU Shading Language) - specyfikacja robocza W3C*. <https://www.w3.org/TR/WGSL/>. [7.1.2026].
- [42] *Wrapper WebGPU dla C++*. <https://github.com/eliemichel/WebGPU-Cpp>. [7.1.2026].

- [43] Wywiad z dyrektorem Allegro. <https://fashionbiznes.pl/moda-online-nad-wisla-allegro-ujawnia-preferencje-zakupowe-polakow-w-2024-roku/>. [15.1.2026].
- [44] Zhang, X., Liu, Y., Qin, L., & Xie, N. (2025). An Cross-Platform Real-Time Rendering Optimization Solution for WebAssembly Combined with Graphic APIs. *2025 8th International Conference on Information and Computer Technologies (ICICT)*, 333–337. <https://doi.org/10.1109/ICICT64582.2025.00058>